

Gate minimisation

Abhilash Jindal

Simplifying boolean functions

- Algebraically simplifying boolean functions is complicated
 - Approach: convert boolean function into its *canonical* form and then apply a mechanical method to simplify
- $xy + x'z + yz$
 - $xy + x'z + yz(x + x')$
 - $x(y + yz) + x'(z + yz)$
 - $xy + x'z$

Minterms

- Call each row of the truth table as a *minterm*, represented by m_i where i represents the decimal equivalent of the binary number
- If you write the truth table of each minterm, notice that it outputs 1 in only its row of the truth table; other rows output 0

			Minterms	
x	y	z	Term	Designation
0	0	0	$x'y'z'$	m_0
0	0	1	$x'y'z$	m_1
0	1	0	$x'yz'$	m_2
0	1	1	$x'yz$	m_3
1	0	0	$xy'z'$	m_4
1	0	1	$xy'z$	m_5
1	1	0	xyz'	m_6
1	1	1	xyz	m_7

Canonical forms

Sum of minterms

- A boolean function can be expressed from a given truth table by forming a minterm for each combination of the variables that produces a 1 in the function and then taking the OR of all those terms.

<i>x</i>	<i>y</i>	<i>z</i>	Function <i>f</i> ₁	Function <i>f</i> ₂
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

- $f_1 = x'y'z + xy'z' + xyz = m_1 + m_4 + m_7$
- $f_2 = x'yz + xy'z + xyz' + xyz = m_3 + m_5 + m_6 + m_7$

Convert a function to sum of minterms from its expression

- $F = A + B'C$
- $A(B + B')(C + C') + (A + A')B'C$
- $ABC + ABC' + AB'C + AB'C' + AB'C + A'B'C$
- $ABC + ABC' + AB'C + AB'C' + A'B'C$
- $A'B'C + AB'C' + AB'C + ABC' + ABC$
- $m_1 + m_4 + m_5 + m_6 + m_7$
- $F(A, B, C) = \Sigma(1,4,5,6,7)$

Table 2.5
Truth Table for $F = A + B'C$

A	B	C	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	1

Maxterms

- Call each row of the truth table as a *maxterm*, represented by M_i where i represents the decimal equivalent of the binary number
- If you write the truth table of each maxterm, notice that it outputs **0** in only its row of the truth table; other rows output **1**

			Maxterms	
x	y	z	Term	Designation
0	0	0	$x + y + z$	M_0
0	0	1	$x + y + z'$	M_1
0	1	0	$x + y' + z$	M_2
0	1	1	$x + y' + z'$	M_3
1	0	0	$x' + y + z$	M_4
1	0	1	$x' + y + z'$	M_5
1	1	0	$x' + y' + z$	M_6
1	1	1	$x' + y' + z'$	M_7

Table 2.4
Functions of Three Variables

<i>x</i>	<i>y</i>	<i>z</i>	Function <i>f</i> ₁	Function <i>f</i> ₂
0	0	0	0	0
0	0	1	1	0
0	1	0	0	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

Canonical forms

Product of maxterms

- A boolean function can be expressed from a given truth table by forming a **maxterm** for each combination of the variables that produces a **0** in the function and then taking the **AND** of all those terms.
- $f_1 = (x + y + z)(x + y' + z)(x + y' + z)(x' + y + z')(x' + y' + z) = M_0M_2M_3M_5M_6$
- $f_2 = (x + y + z)(x + y + z')(x + y' + z)(x' + y + z) = M_0M_1M_2M_4$

Minterms and Maxterms are complements

- $m'_j = M_j$

- $M'_j = m_j$

			Minterms		Maxterms	
x	y	z	Term	Designation	Term	Designation
0	0	0	$x'y'z'$	m_0	$x + y + z$	M_0
0	0	1	$x'y'z$	m_1	$x + y + z'$	M_1
0	1	0	$x'yz'$	m_2	$x + y' + z$	M_2
0	1	1	$x'yz$	m_3	$x + y' + z'$	M_3
1	0	0	$xy'z'$	m_4	$x' + y + z$	M_4
1	0	1	$xy'z$	m_5	$x' + y + z'$	M_5
1	1	0	xyz'	m_6	$x' + y' + z$	M_6
1	1	1	xyz	m_7	$x' + y' + z'$	M_7

Conversion between canonical forms

- $F(A, B, C) = \Sigma(1,4,5,6,7)$
- $F'(A, B, C) = \Sigma(0,2,3) = m_0 + m_2 + m_3$
- $F(A, B, C) = m'_0 m'_2 m'_3$
- $F(A, B, C) = M_0 M_2 M_3$
- $F(A, B, C) = \Pi(0,2,3)$

Interchange Σ and Π and list the numbers missing from the original form

Conversion between canonical forms

Why does the approach work?

- $F = xy + x'z$
- Find all ones from the truth table
 - $F = \Sigma(1,3,6,7)$
- Remaining have zeros in the truth table
 - $F = \Pi(0,2,4,5)$

Truth Table for $F = xy + x'z$

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	0
1	1	0	1
1	1	1	1

Minterms

Maxterms

Exercise: Identify minterms and maxterms

- $F = \Sigma(1,3,4,6) = \Pi(0,2,5,7)$
- Any Boolean function can be *uniquely* written in its canonical forms: sum-of-minterms or product-of-maxterms form

x	y	z	F
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	0

Standard forms

Sum of products

- “Mechanical method” will convert from canonical form to standard form, i.e., sum of minterms to sum of products
 - Literals: x , x' , y , y' , z , z' etc
 - Product terms: y' , xy , $x'yz'$
 - Sum of product: $y' + xy + x'yz'$

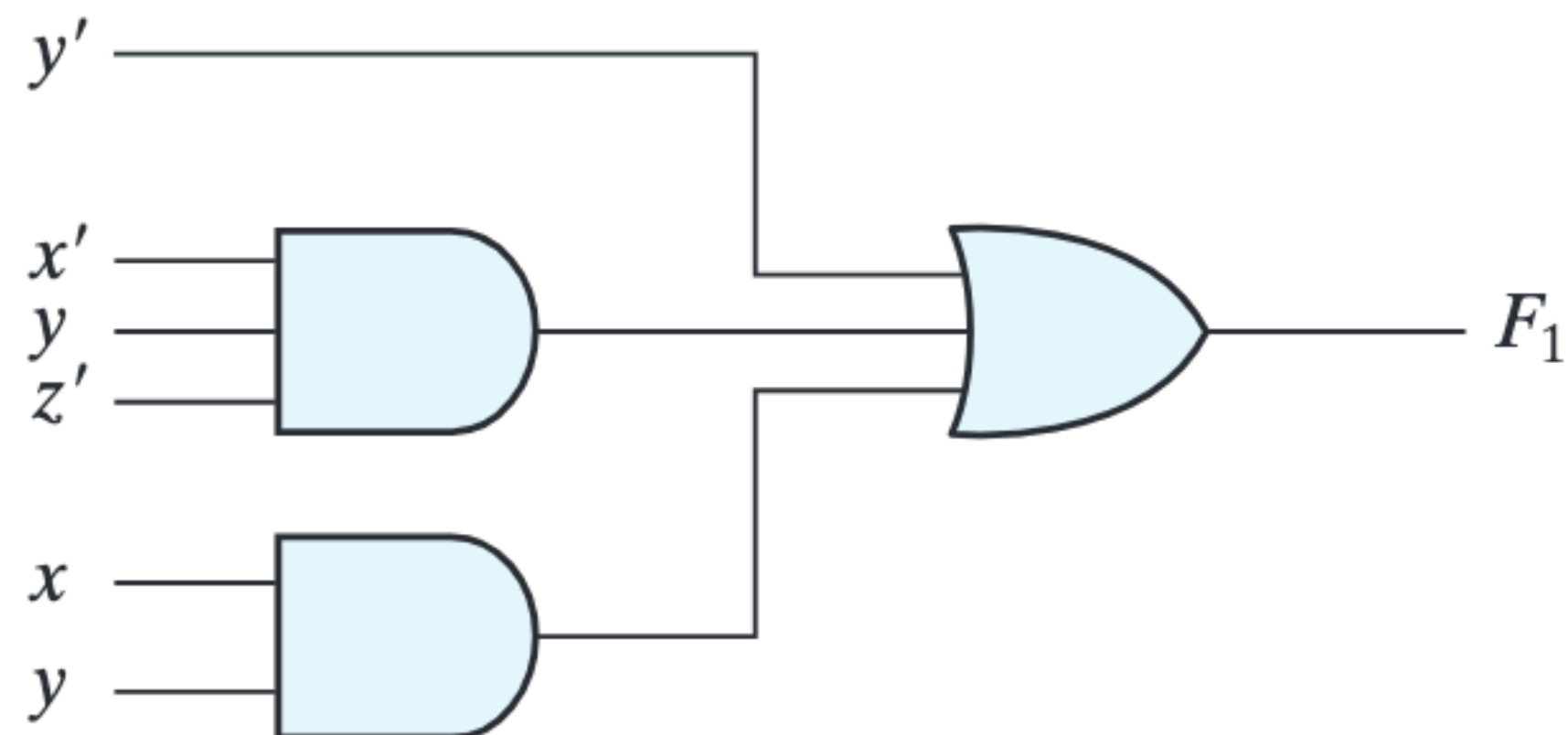
Standard forms

Product of sums

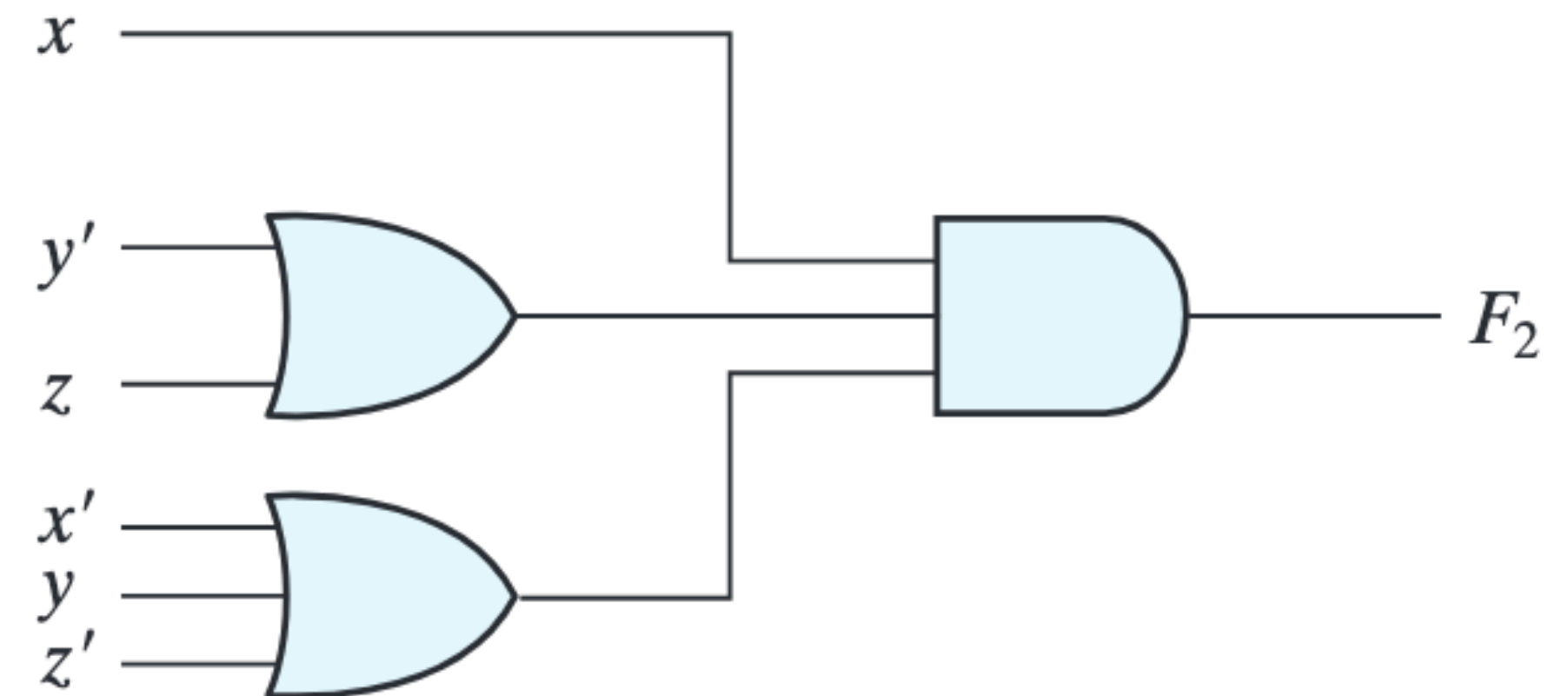
- “Map method” will convert from canonical form to standard form, i.e., product of maxterms to product of sums
 - Literals: x , x' , y , y' , z , z' etc
 - Product terms: x , $y'+z$, $x' + y + z'$
 - Sum of product: $x (y'+z) (x'+y+z')$

Standard forms lead to two-level structure of gates

- Preferred because it produces least amount of delay through the gates when the signal propagates from the input to output



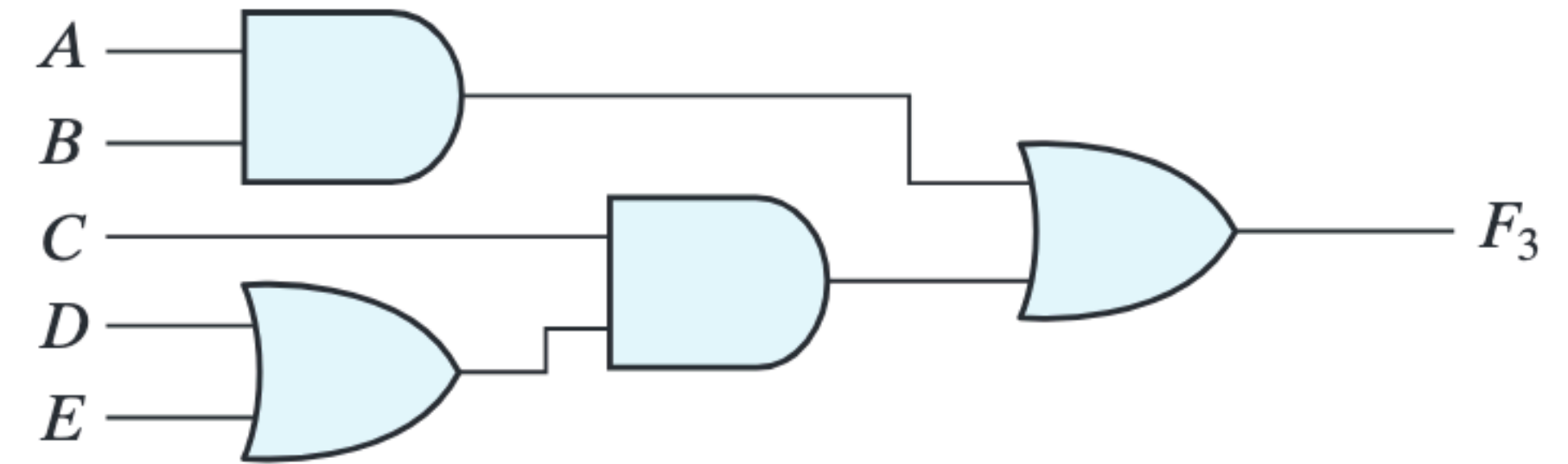
(a) Sum of Products



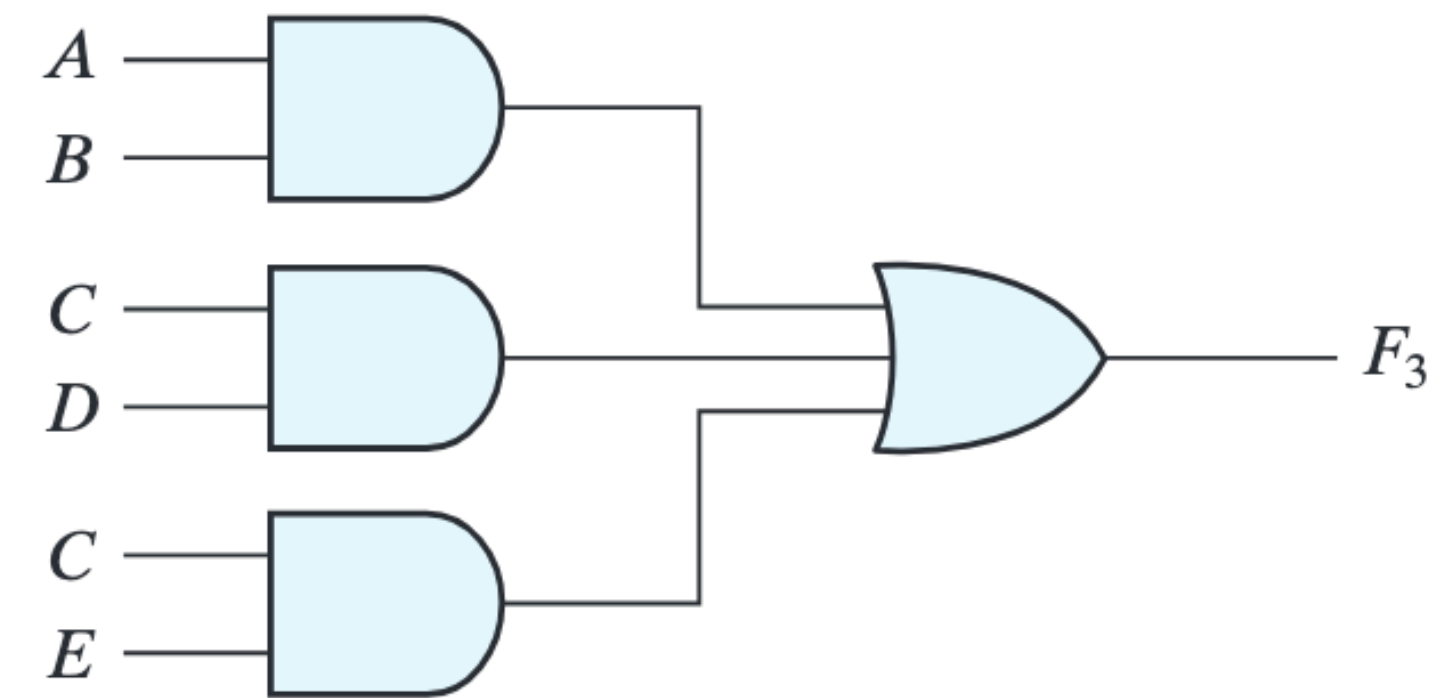
(b) Product of Sums

Non-standard forms lead to larger delays

- $F = AB + C(D + E)$
- $F = AB + CD + CE$



(a) $AB + C(D + E)$



(b) $AB + CD + CE$

Gate-level minimisation

- Find an optimal gate-level implementation of the Boolean function
- For a given function, truth table is unique, but when expressed algebraically it can appear in many equivalent forms
 - $xy + x'z + yz$
 - $xy + x'z + yz(x + x')$
 - $x(y + yz) + x'(z + yz)$
 - $xy + x'z$
- Expressions can be simplified algebraically
 - Lacks rules to predict each succeeding step
 - Unclear if we reached a “minimal function”
 - Least number of inputs/gates
 - Minimum number of terms with minimum number of literals in each term

Karnaugh map method

- Any Boolean function can be uniquely represented as sum of minterms
- Represent each Boolean function graphically
- Follow straightforward rules to recognise patterns
- Patterns give alternative algebraic expressions for the same function. Select the simplest one
- Guaranteed to produce a minimal expression: minimum number of terms with minimum number of literals in each term

Two-variable K-map

0	0	m_0
0	1	m_1
1	0	m_2
1	1	m_3

		y	
		0	1
x	0	m_0 $x'y'$	m_1 $x'y$
	1	m_2 xy'	m_3 xy

- Four squares, one for each minterm
- 0 and 1 marked in each row and column designate the values of the variables.
- $x=0$ in first row. $x=1$ in second row. $y=0$ in first column. $y=1$ in second column

Representing functions in the K-map

- Basically a pictorial representation of the truth table
- xy
 - Equal to minterm 3 (= 11)
- Mark the minterms with 1 where the function outputs 1

x	y	$x \cdot y$
0	0	0
0	1	0
1	0	0
1	1	1

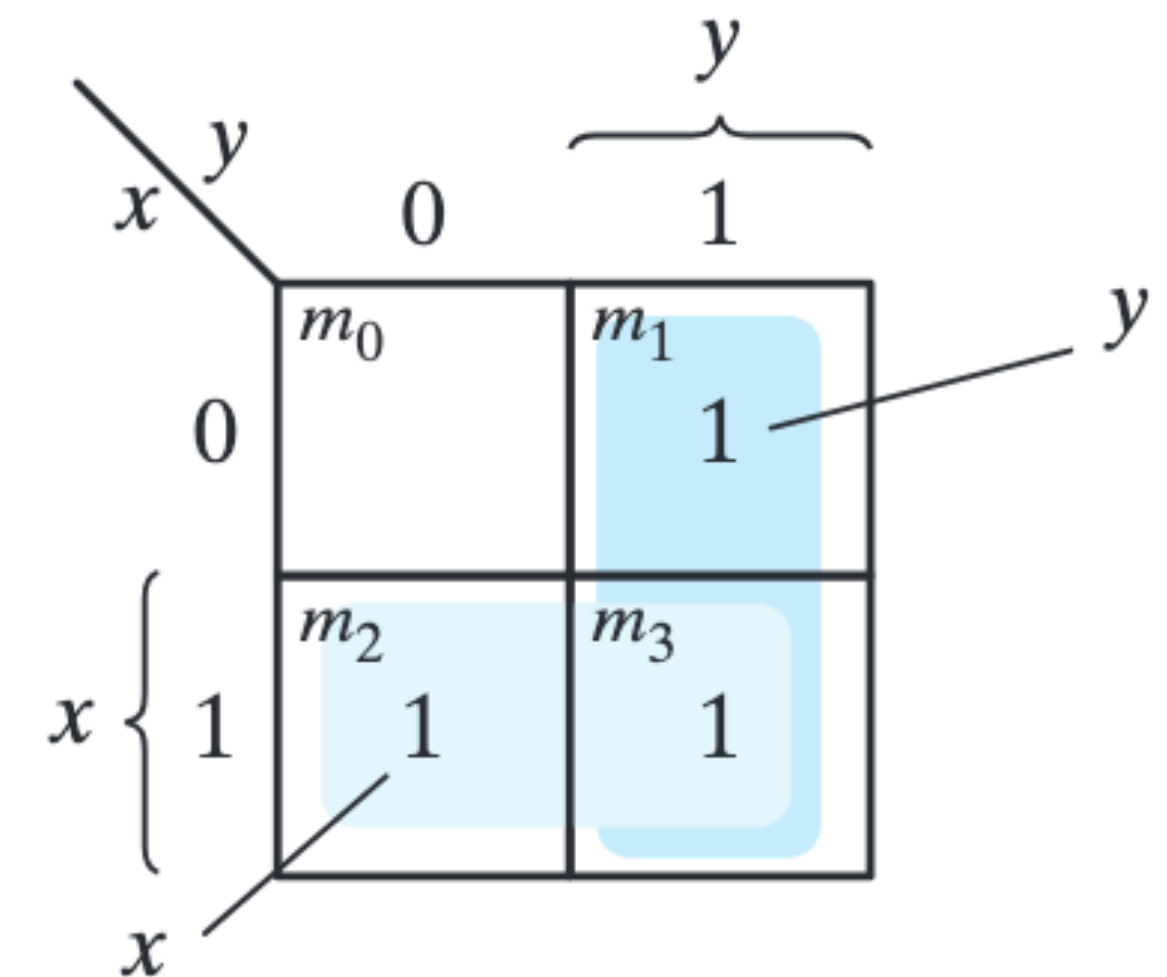
		y	
		0	1
x	0	m_0	m_1
	1	m_2	m_3 1

(a) xy

Representing functions in the K-map

- $x+y$
 - $m_1 + m_2 + m_3$
- The three squares could also be determined as the union of the squares of variable x in the second row and those of variable y in the second column

x	y	$x + y$
0	0	0
0	1	1
1	0	1
1	1	1



(b) $x + y$

Three-variable K-map

$F(x, y, z)$

- m_5 (=101) is row 1 and column 01, i.e., $xy'z$
- When a variable equals 1, there are four corresponding squares

		y			
		yz		11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

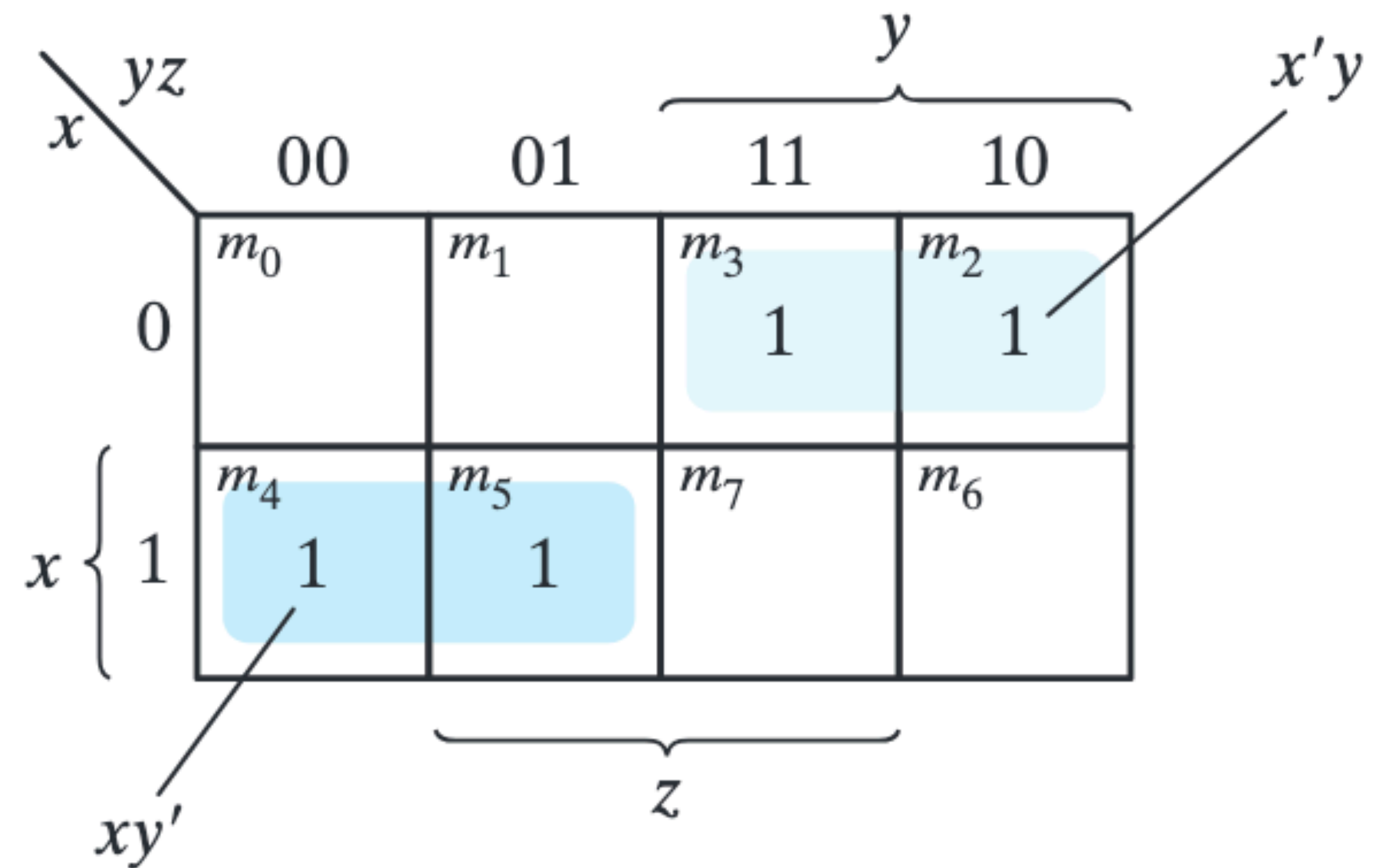
Usefulness of the map

- Only 1-bit changes between any two adjacent locations i.e., only one variable is primed in one square and unprimed in another, other variables are identical.
 - y' in m_5 and y in m_7 . Both have xz .
- Sum of two adjacent minterms will be one product term
 - $m_5 + m_7 = xy'z + xyz = xz(y + y') = xz$
- Any two minterms in adjacent squares (vertically or horizontally) when OR'ed together will remove the dissimilar variable

		y				
		yz		00	01	11
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$	
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'	
		z				

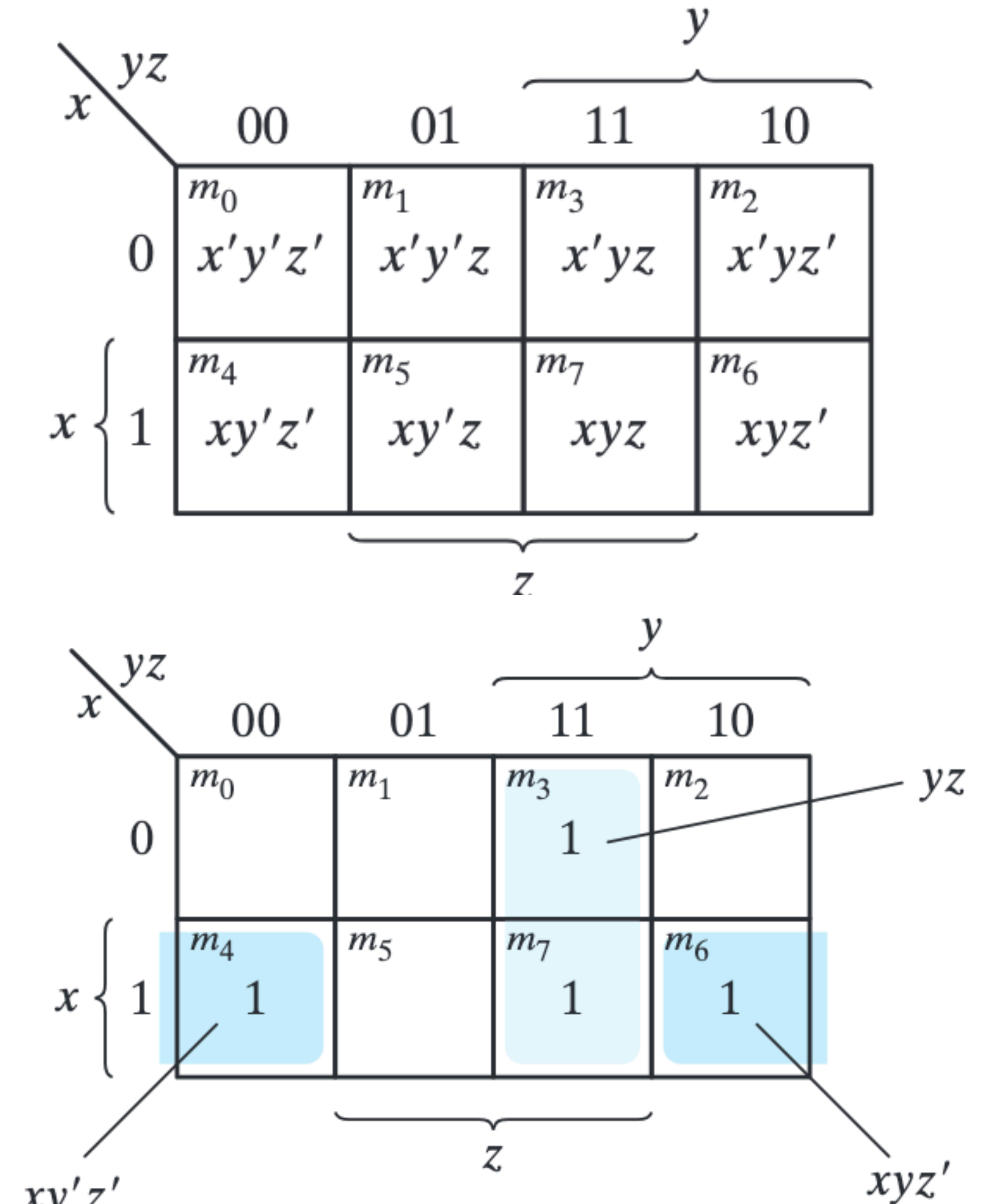
Simplifying functions

- $F(x, y, z) = \Sigma(2,3,4,5)$
- $F(x, y, z) = x'y + xy'$



Adjacent squares

- m_0 is adjacent to m_2 and m_4 is adjacent to m_6 because their minterms differ by only one variable.
- Simplify $\Sigma(3,4,6,7)$
 - $yz + xz'$



Note: $xy'z' + xyz' = xz'$

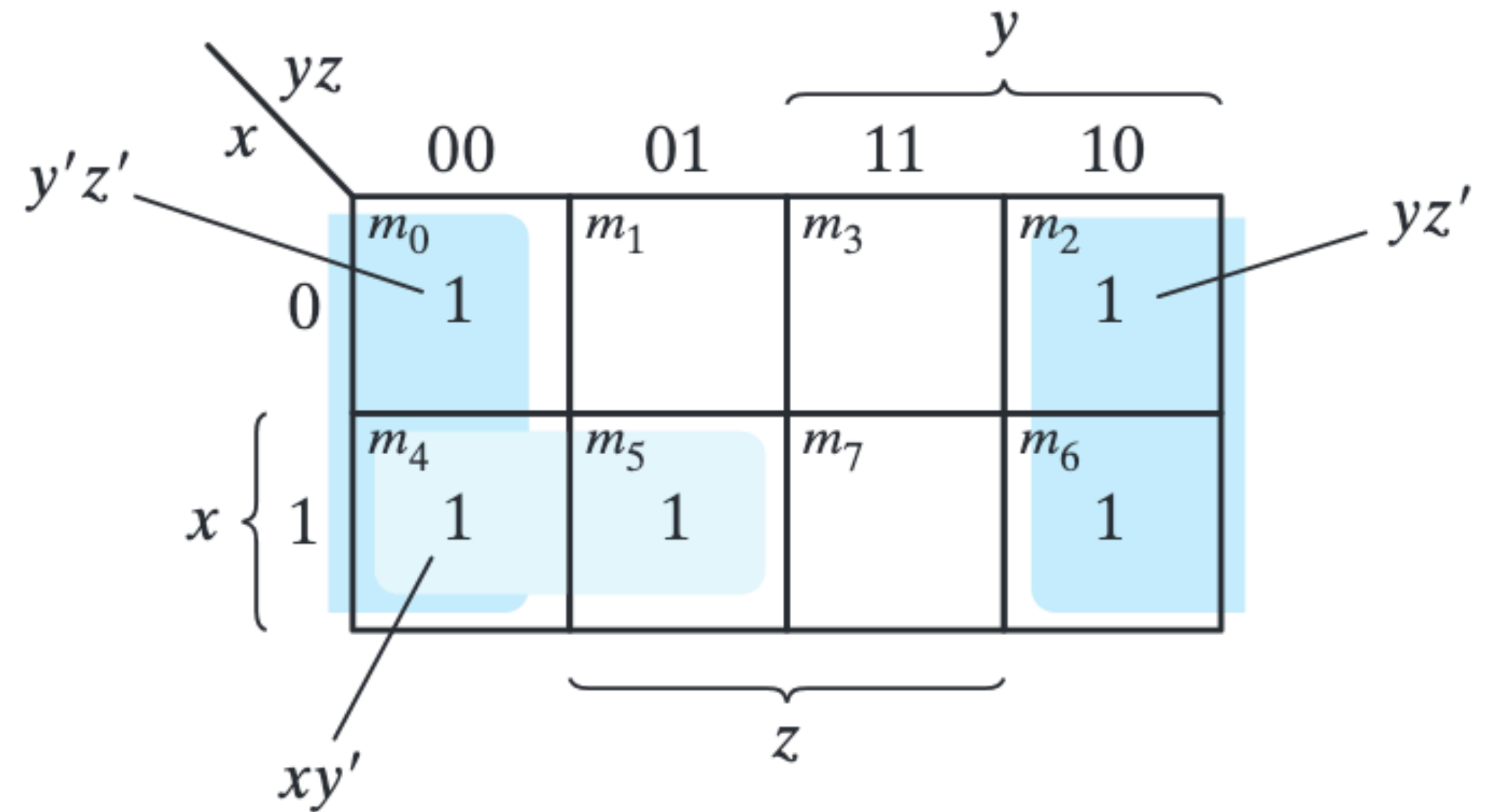
Four adjacent squares

- Any combination of four minterms results in an expression with only one literal
- $m_0 + m_2 + m_4 + m_6 =$
 - $x'y'z' + x'yz' + xy'z' + xyz'$
 - $x'z'(y' + y) + xz'(y' + y)$
 - $x'z' + xz'$
 - z'

		y			
		yz			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'

Simplify the Boolean function

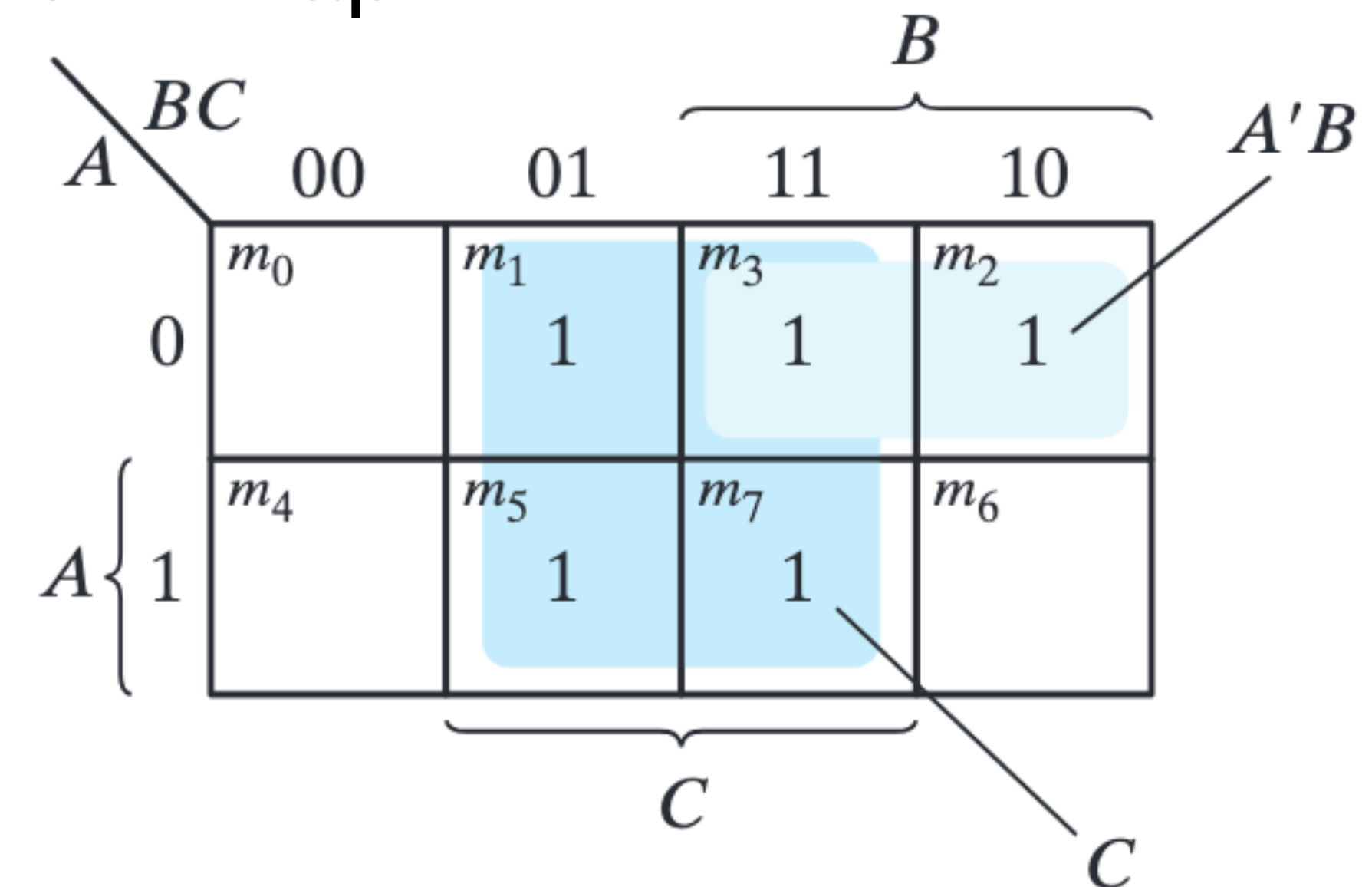
- $F(x, y, z) = \Sigma(0, 2, 4, 5, 6)$
- $z' + xy'$
- Notice that m_5 is combined with an already used square m_4
- This is desirable as it represents xy' whereas the single m_5 square represents $xy'z$



Note: $y'z' + yz' = z'$

Simplify sum of products

- Function not in sum-of-minterms form: $F = A'C + A'B + AB'C + BC$
- Put 1 in all the squares corresponding to $A'C$; $A'B$; $AB'C$; BC
- Function has 5 minterms as it has 5 1's in the K-map
- $F = C + A'B$



Minimisation Approach

Combine adjacent squares in powers of two

- One square represents one minterm, giving a term with three literals
- Two adjacent squares represent a term with two literals
- Four adjacent squares represent a term with one literal
- Eight adjacent squares
 - Encompass the entire three-variable map and produces a function that is always equal to 1

Minimisation Approach (2)

- Cover all the minterms when we combine the squares
- Minimize the number of terms in the expression (try to combine more squares)
- There are no redundant terms (i.e., minterms already covered by other terms)
- Exercise: $F(x, y, z) = \Sigma(0, 2, 3, 4, 6)$
 - $z' + x'y$

Four-variable K-map

$$F(w, x, y, z)$$

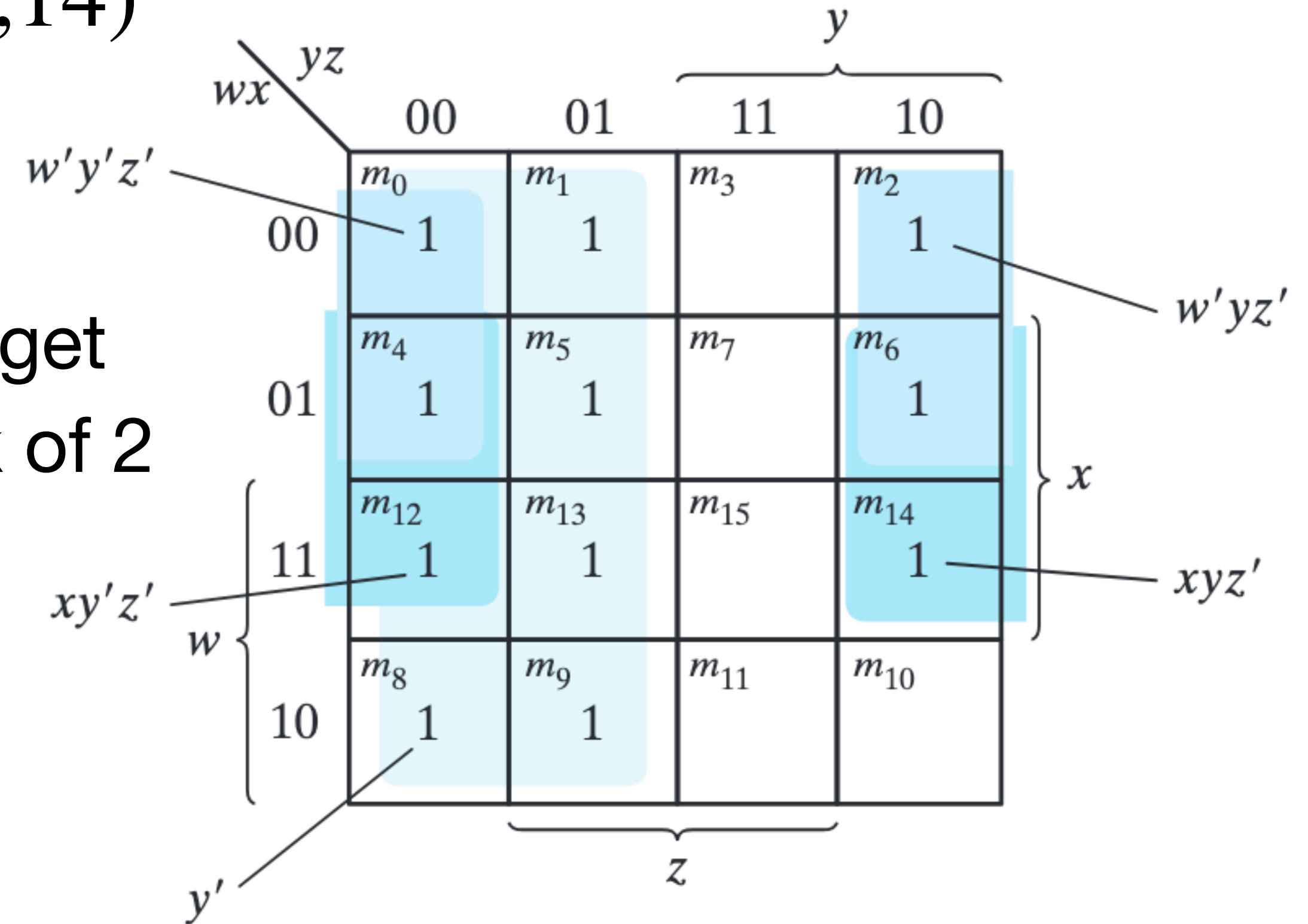
- 13 is 1101. m_{13}
- Notice that in any two adjacent squares only one variable is primed in one square and unprimed in another; all other variables are identical
- m_0 and m_2 are adjacent
- m_3 and m_{11} are adjacent

		y			
		yz		11	10
w	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$

Minimisation approach

Combine adjacent squares in powers of two

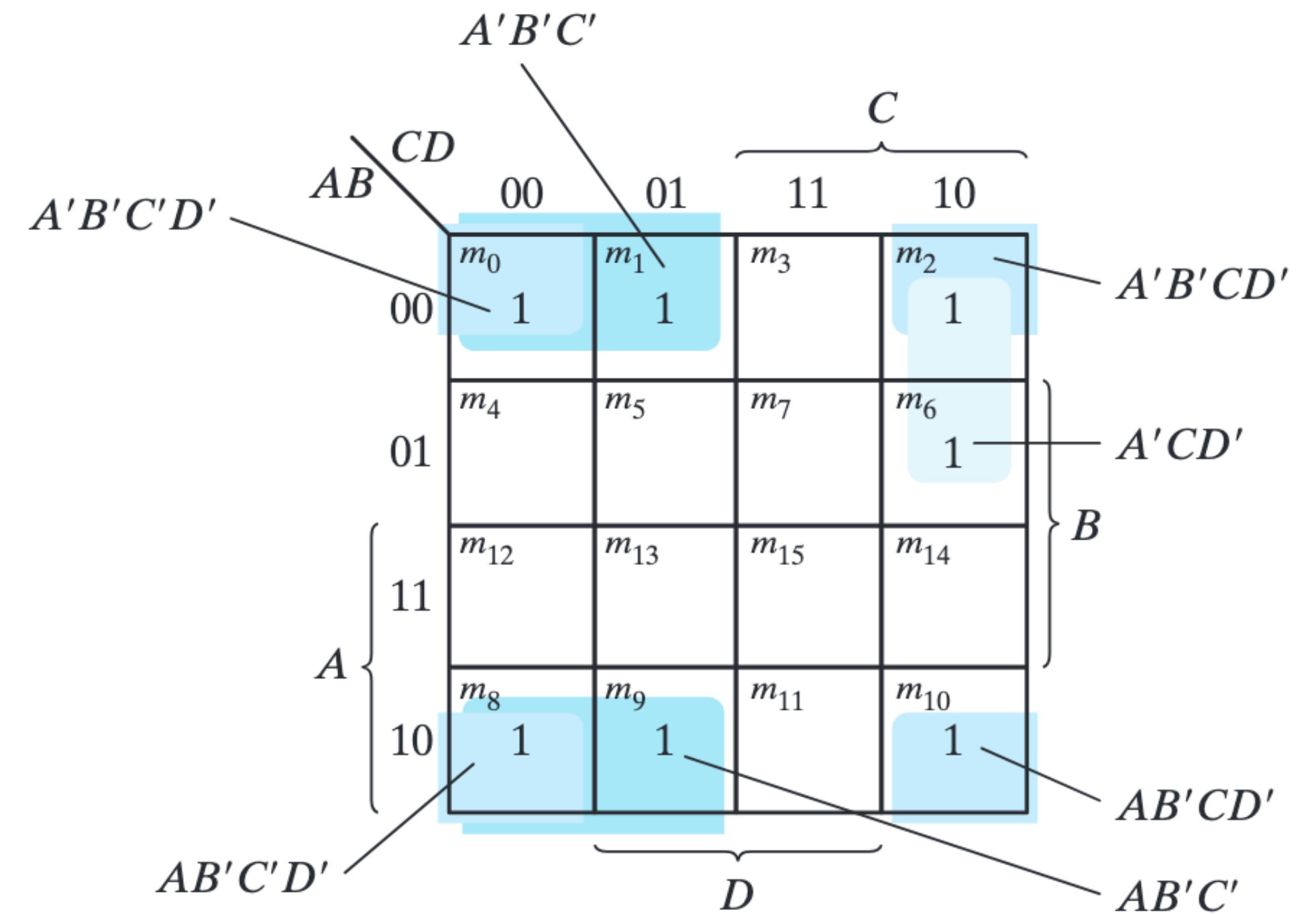
- $F(w, x, y, z) = \Sigma(0,1,2,4,5,6,8,9,12,13,14)$
- $= y' + w'z' + xz'$
- Prefer combining m_2 into a block of 4 to get less literals: $w'z'$. Combining into a block of 2 would have resulted in $w'yz'$



Note: $w'y'z' + w'yz' = w'z'$
 $xy'z' + xyz' = xz'$

Simplify

- $F = A'B'C' + B'CD' + A'BCD' + AB'C'$
- Mark all 1's. Example: $A'B'C'$ marks m_0 and m_1
- All the four corners can be combined into $B'D'$
- $F = B'D' + B'C' + A'CD'$



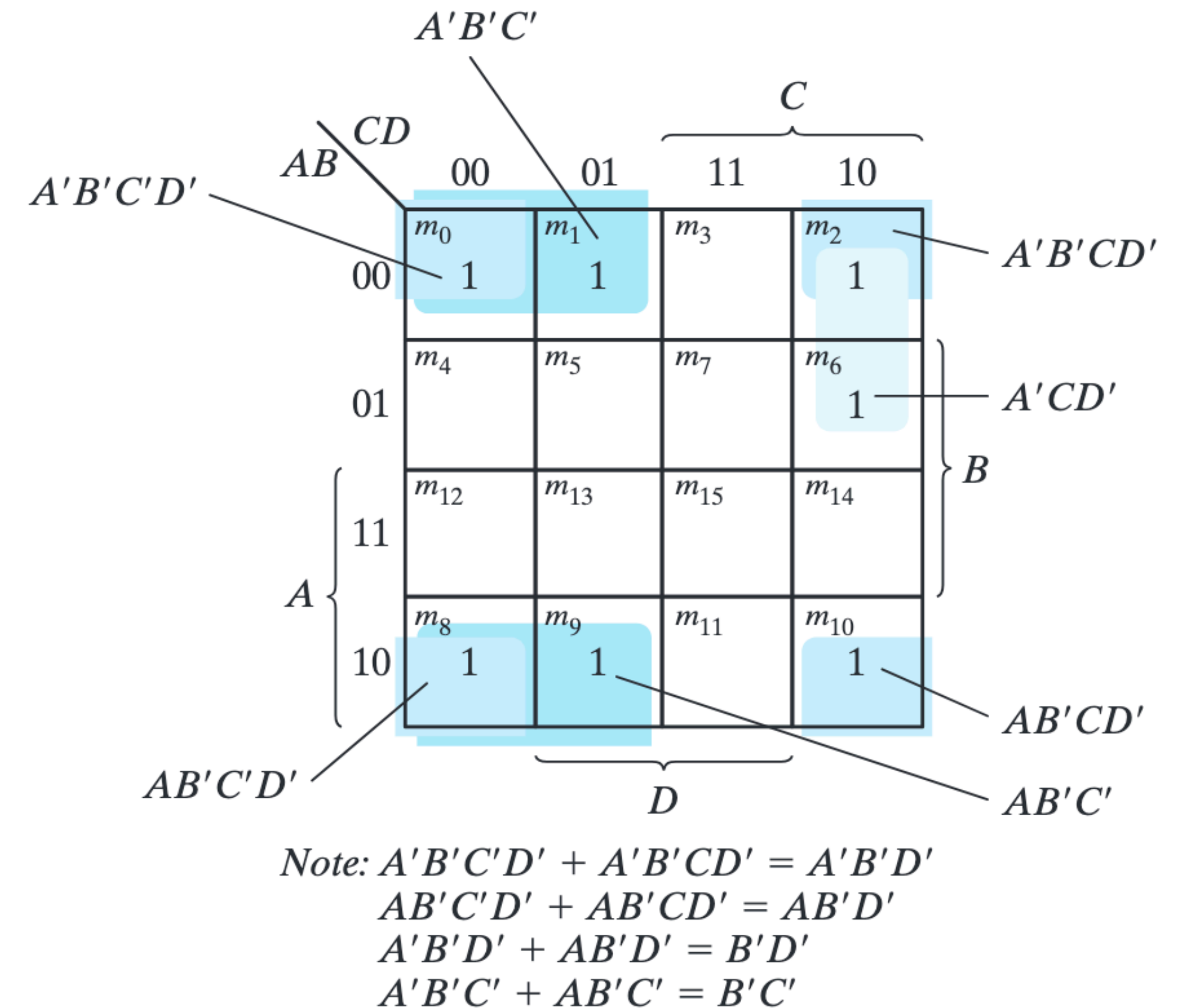
Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$
 $A'B'C' + AB'C' = B'C'$

Expression can have many minimal simplifications

- $F = \Sigma(0,2,3,5,7,8,9,10,11,13,15)$
- $BD + B'D' + CD + AD$
- $BD + B'D' + CD + AB'$
- $BD + B'D' + B'C + AD$
- $BD + B'D' + B'C + AB'$

Implicants

- Product terms of a sum of products expression are called *implicants* of the function. The function has a value of 1, when the implicant is 1.



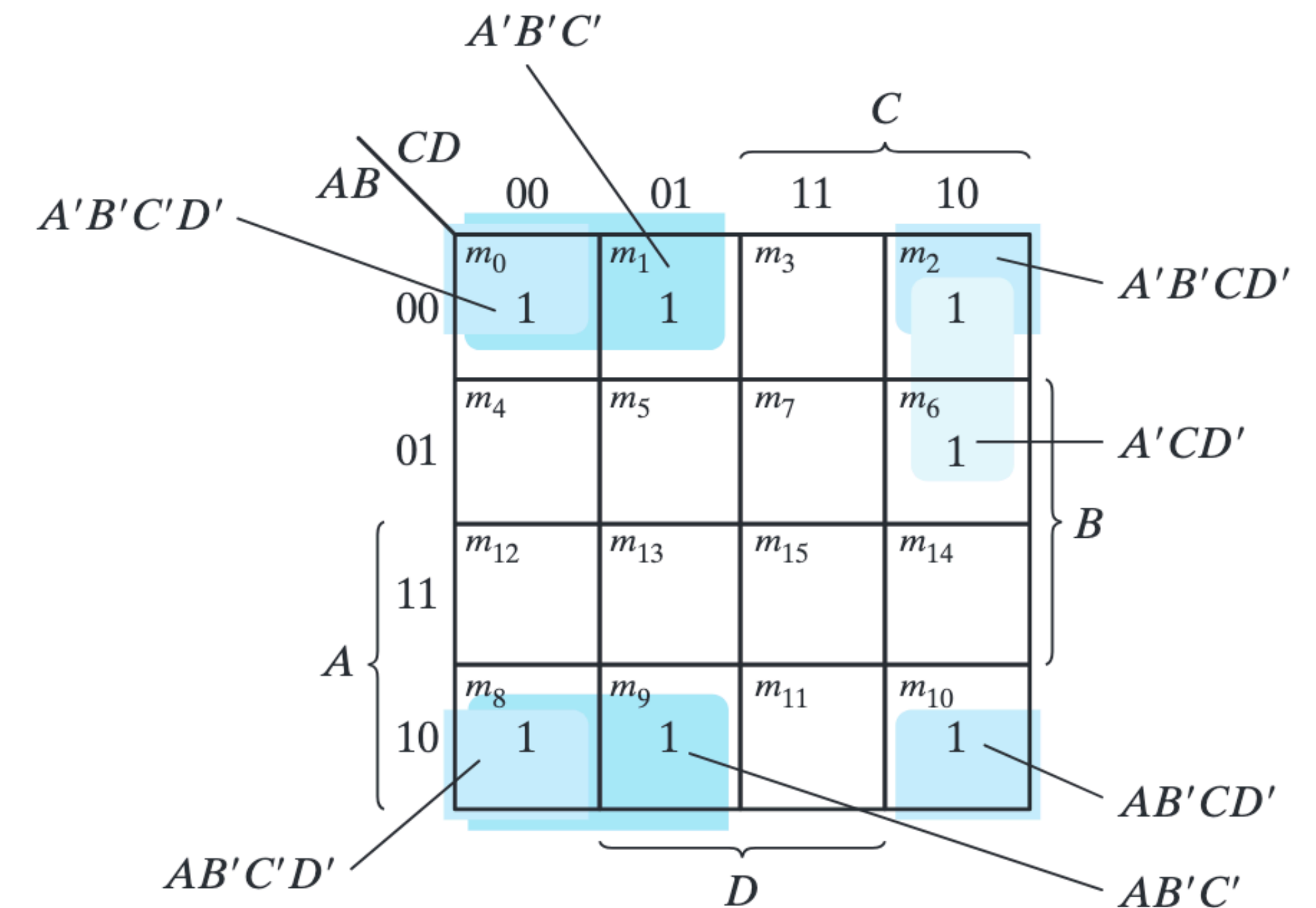
- Example:

- $$F = A'B'C' + B'CD' + A'BCD' + AB'C' = B'D' + B'C' + A'CD'$$

- F is 1 when $A'B'C'$ is 1; when $B'D'$ is 1; when $B'C'$ is 1, ...

Prime implicants

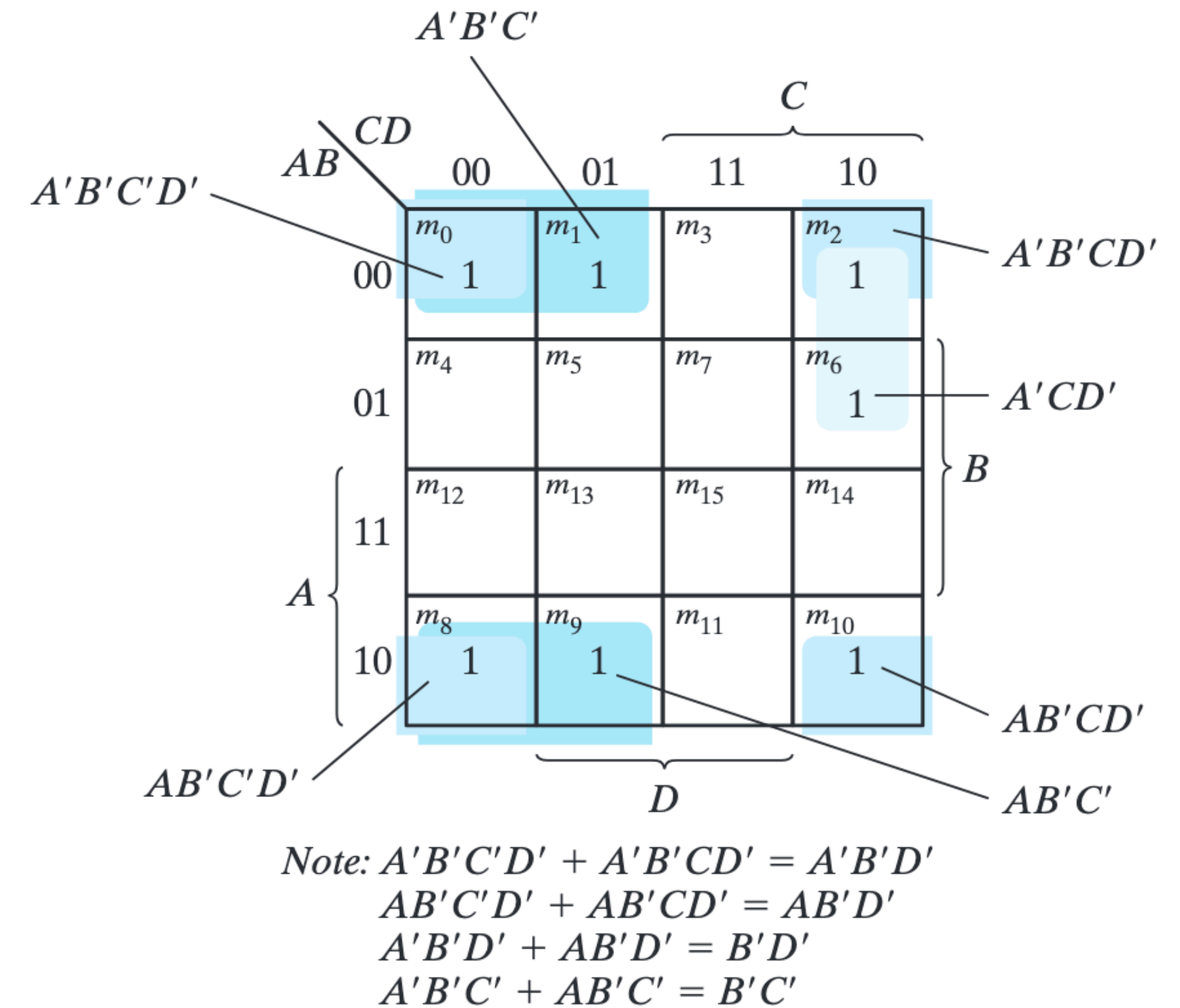
- A *prime implicant* is a product term obtained by combining the maximum possible number of adjacent squares in the map.
- $F = A'B'C' + B'CD' + A'BCD' + AB'C' = B'D' + B'C' + A'CD'$
- $A'BCD'$ is not a prime implicant
- $A'CD'$ is a prime implicant



Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$
 $A'B'C' + AB'C' = B'C'$

Prime implicants

- A single 1 represents a prime implicant if it is not adjacent to any other 1's
- Two adjacent 1's form a prime implicant if it is not within a group of four adjacent squares. And so on.
- $A'CD'$ is a prime implicant



Essential prime implicants

- A prime implicant is *essential* if it is the only prime implicant that covers a minterm

- $F = \Sigma(0,2,3,5,7,8,9,10,11,13,15)$

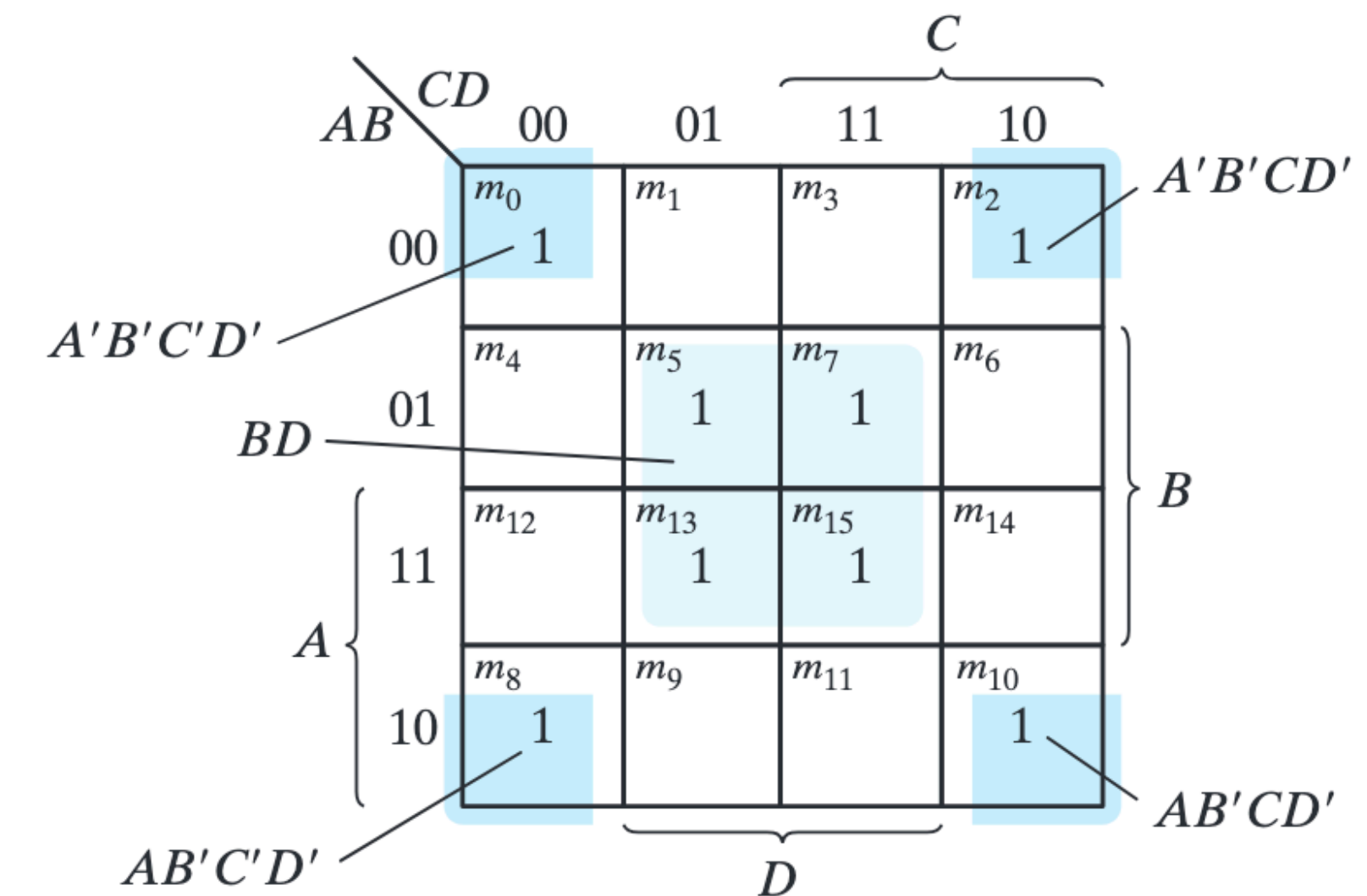
- $BD + B'D' + CD + AD$

- $BD + B'D' + CD + AB'$

- $BD + B'D' + B'C + AD$

- $BD + B'D' + B'C + AB'$

- BD and $B'D'$ are essential prime implicants

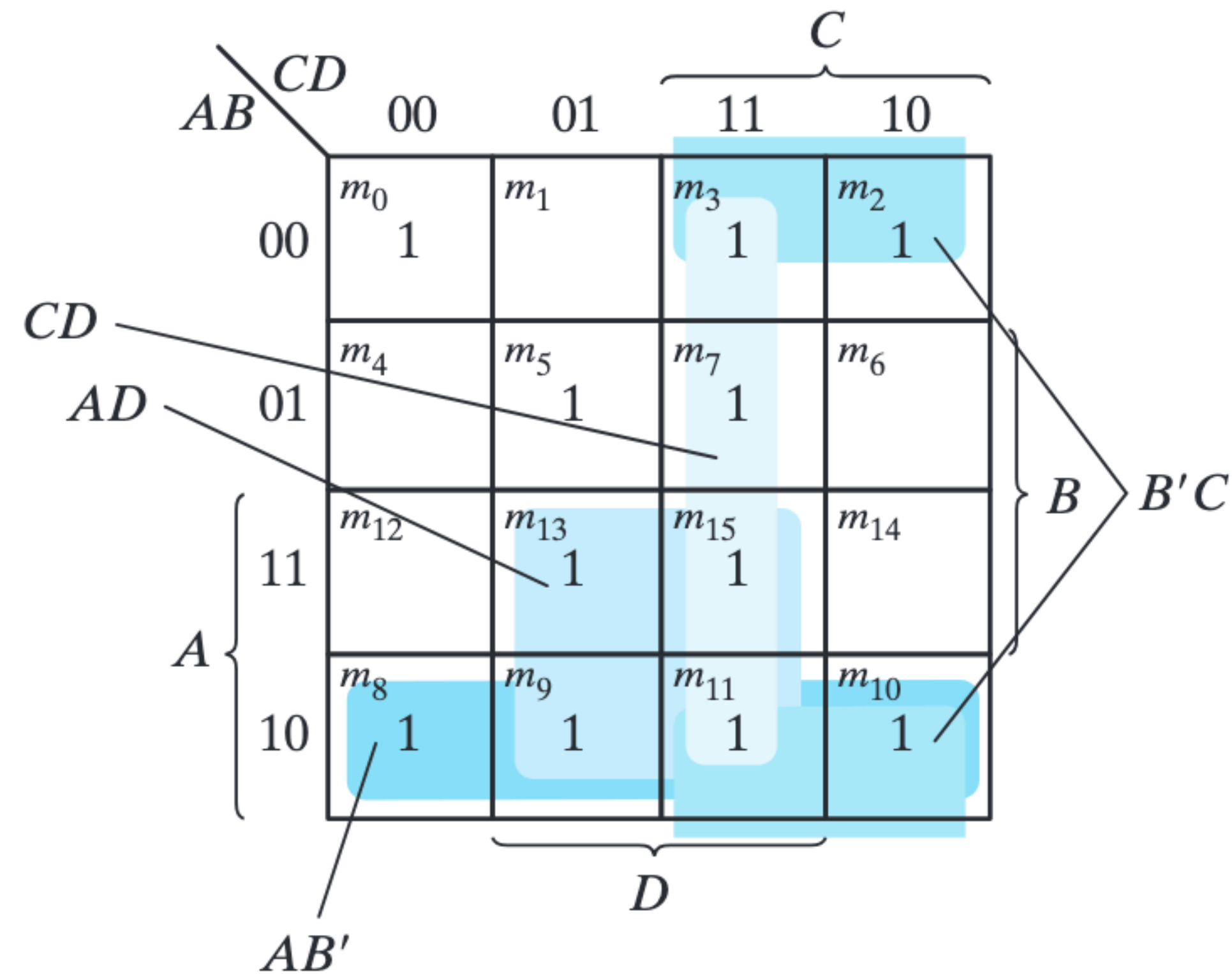


Note: $A'B'C'D' + A'B'CD' = A'B'D'$
 $AB'C'D' + AB'CD' = AB'D'$
 $A'B'D' + AB'D' = B'D'$

(a) Essential prime implicants
 BD and $B'D'$

Essential prime implicants

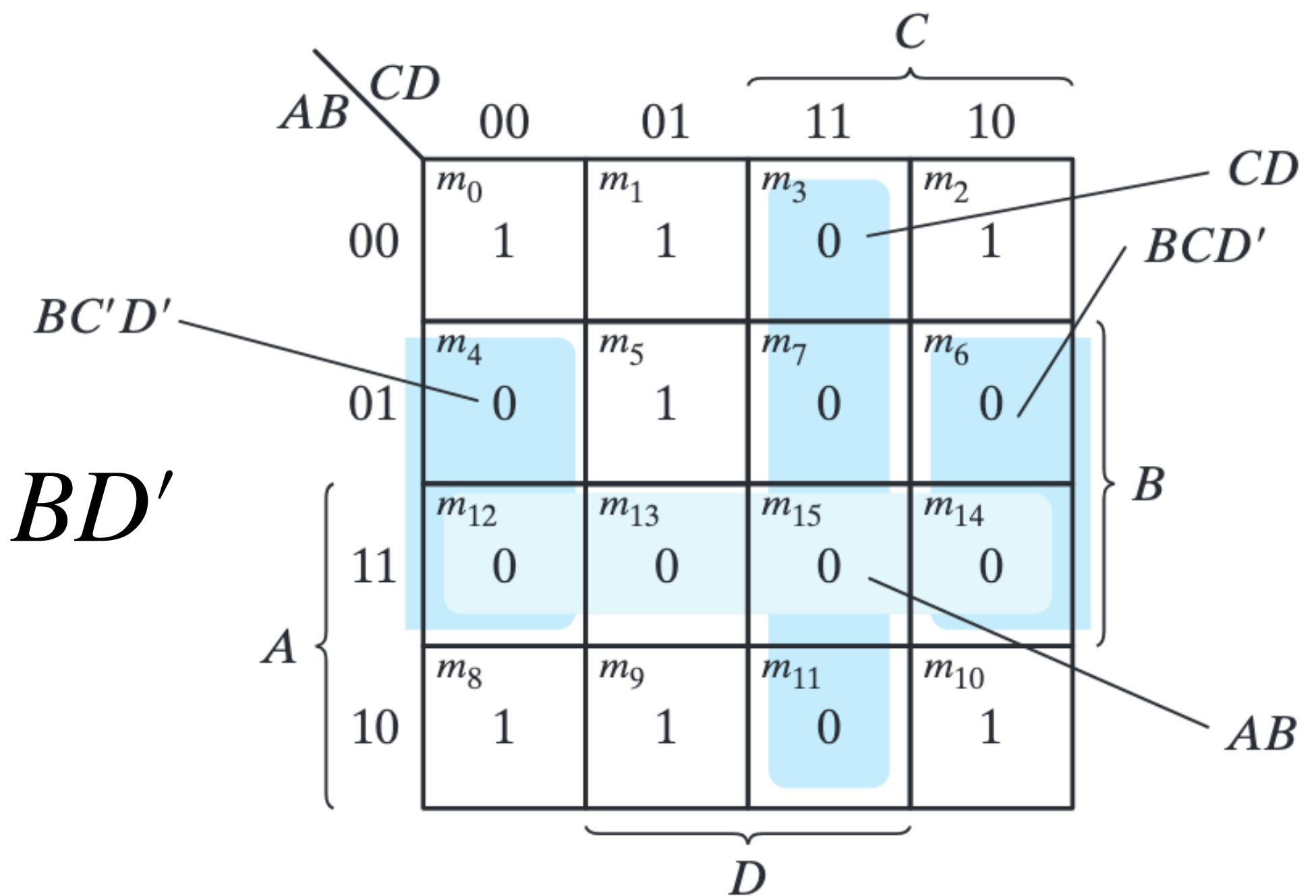
- $F = \Sigma(0,2,3,5,7,8,9,10,11,13,15)$
- First find essential prime implicants. Cover remaining minterms by prime implicants
- m_3 can be covered by both CD and $B'C$
- m_9 can be covered by both AD and AB'



(b) Prime implicants CD , $B'C$, AD , and AB'

Product-of-sums simplification

- $F(A, B, C, D) = \Sigma(0,1,2,5,8,9,10)$
- Mark the 0s in K-map F'
- Combine the squares $F' = AB + CD + BD'$
- $F = (A' + B')(C' + D')(B' + D)$



Note: $BC'D' + BCD' = BD'$

Product-of-sums simplification (2)

- $F(x, y, z) = \Pi(0,2,5,7)$
- Write 0s in K-map and merge
- $F' = xz + x'z'$
- $F = (x' + z')(x + z)$

Don't care conditions

Incompletely-specified function

- Sometimes the function is not specified for certain combination of variables
- Example: say we have a four-bit input and we have to show digits 0-9 on a seven-segment display, i.e., there are seven functions $F_{0-6}(w, x, y, z)$
 - Function value 1010 to 1111 is not specified
 - Minterms m_{10} to m_{15} are *don't care minterms*

Binary-Coded Decimal (BCD)

Decimal Symbol	BCD Digit
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Truth table with don't care conditions

- $F(w, x, y, z) = \Sigma(1,3,7,11,15)$
- $d(w, x, y, z) = \Sigma(0,2,5)$

w	x	y	z	F
0	0	0	0	x
0	0	0	1	1
0	0	1	0	x
0	0	1	1	1
0	1	0	0	0
0	1	0	1	x
0	1	1	0	0
0	1	1	1	1
1	0	0	0	0
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

K-map with don't care conditions

- $F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$
 $d(w, x, y, z) = \Sigma(0, 2, 5)$
- Mark minterms of F with 1 and those of d with X. Remaining are zero.
- X can be considered as either 0 or 1.
- Must include all five 1's in the K-map, but we may or may not include any of the X's depending on the way function is simplified.

		y			
		00	01	11	10
w	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0
		z			

(a) $F = yz + w'x'$

Simplifying incompletely specified functions

- $F(w, x, y, z) = \Sigma(1,3,7,11,15)$
 $d(w, x, y, z) = \Sigma(0,2,5)$

- An incompletely specified function can be simplified into different functions

- yz is an essential prime implicant

		y			
	yz	00	01	11	10
wx	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0
w					

(a) $F = yz + w'x'$

		y			
	yz	00	01	11	10
wx	00	m_0 X	m_1 1	m_3 1	m_2 X
	01	m_4 0	m_5 X	m_7 1	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 1	m_{14} 0
	10	m_8 0	m_9 0	m_{11} 1	m_{10} 0
w					

(b) $F = yz + w'z$

Simplifying incompletely specified functions (2)

- An incompletely specified function can be simplified into two different functions which are *algebraically not-equal*.
- $F(w, x, y, z) = \Sigma(1,3,7,11,15)$ $d(w, x, y, z) = \Sigma(0,2,5)$
 - $F_1(w, x, y, z) = yz + w'x' = \Sigma(0,1,2,3,7,11,15)$
 - $F_2(w, x, y, z) = yz + w'z' = \Sigma(1,3,5,7,11,15)$
 - Both of them must include the original minterms $\Sigma(1,3,7,11,15)$
 - Differ in the don't care minterms. F_1 assigns 1 to m_0, m_2 . F_2 assigns 1 to m_5

Simplifying into product of sum

- $F(w, x, y, z) = \Sigma(1,3,7,11,15)$ $d(w, x, y, z) = \Sigma(0,2,5)$
- Only one-way to combine the 0's into prime implicants
- $F'(w, x, y, z) = z' + wy'$
- Both z' and wy' are essential prime implicants for F'
- $F(w, x, y, z) = z(w' + y)$

Simplify the Function

- $F(w, x, y, z) = \Sigma(4,5,6,7,12)$
- $d(w, x, y, z) = \Sigma(0,8,13)$
- $F(w, x, y, z) = xy' + xw'$

Simplifying functions with more variables (1)

Quine McCluskey Method

- Same idea as Karnaugh-map but applied using a tabular approach instead of a graphical approach. More suitable for implementation
- Step 0: start with a function in its canonical representation: sum-of-minterms
- $F(A, B, C, D) = \Sigma(0,1,2,4,6,8,9,11,13,15)$

Simplifying functions with more variables (2)

Quine McCluskey Method

- $F(A, B, C, D) = \Sigma(0,1,2,4,6,8,9,11,13,15)$
- Step 1: Convert given minterms to their binary representation and arrange them according to the number of ones present in the binary representation

Group	Minterm	A	B	C	D
0	0	0	0	0	0
1	1	0	0	0	1
	2	0	0	1	0
	4	0	1	0	0
	8	1	0	0	0
2	6	0	1	1	0
	9	1	0	0	1
3	11	1	0	1	1
	13	1	1	0	1
4	15	1	1	1	1

Simplifying functions with more variables (3)

Quine McCluskey Method

- $F(A, B, C, D) = \Sigma(0,1,2,4,6,8,9,11,13,15)$
- Step 2: Compare adjacent groups and combine terms that differ by one bit. Replace the differing bit with - (don't care)

Group	Pair	A	B	C	D
0	(0,1)	0	0	0	-
	(0,2)	0	0	-	0
	(0,4)	0	-	0	0
	(0,8)	-	0	0	0
1	(1,9)	-	0	0	1
	(2,6)	0	-	1	0
	(4,6)	0	1	-	0
	(8,9)	1	0	0	-
2	(9,11)	1	0	-	1
	(9,13)	1	-	0	1
3	(11,15)	1	-	1	1
	(13,15)	1	1	-	1

Simplifying functions with more variables (4)

Quine McCluskey Method

- $F(A, B, C, D) = \Sigma(0,1,2,4,6,8,9,11,13,15)$
- Step 3: Repeat the combining process until no further combinations are possible.

Group	Quad	A	B	C	D
0	(0,1,8,9)	-	0	0	-
	(0,2,4,6)	0	-	-	0
2	(9,11,13,15)	1	-	-	1

Simplifying functions with more variables (5)

Quine McCluskey Method

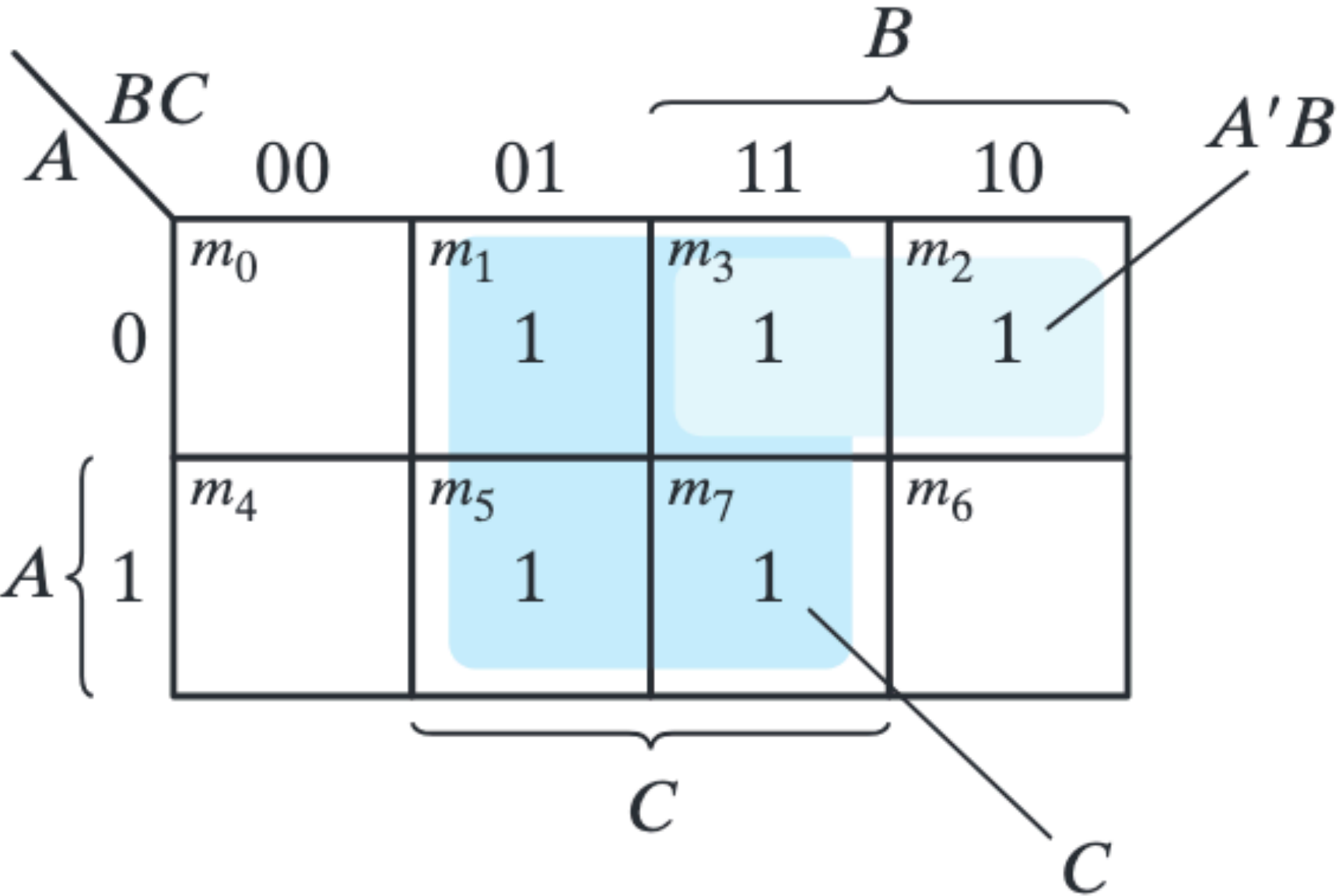
- When no further simplification is possible, we end up with prime implicants
- If a minterm is covered by only prime-implicant, that is an essential prime implicant
- $F = \Sigma(0,1,2,4,6,8,9,11,13,15)$
- $F = B'C' + A'D' + AD$

Prime implicants	Minterms									
	0	1	2	4	6	8	9	11	13	15
B'C' (0, 1, 8, 9)	1	1				1	1			
A'D' (0, 2, 4, 6)	1		1	1	1					
AD (9, 11, 13, 15)							1	1	1	1

Quine McCluskey Method

Identical to K-map. Visual vs tabular

• $F(A, B, C) = \Sigma(1,2,3,5,7)$



Group	Minterm	A B C
Group 1 (one 1)	1	0 0 1
	2	0 1 0
Group 2 (two 1's)	3	0 1 1
	5	1 0 1
Group 3 (three 1's)	7	1 1 1

Group	Pairs	A B C
1	(1, 3)	0 - 1
	(1, 5)	- 0 1
	(2, 3)	0 1 -
2	(3, 7)	- 1 1
	(5, 7)	1 - 1

Quad	A B C
(1, 3, 5, 7)	- - 1

Prime Implicant	1	2	3	5	7
C (1,3,5,7)	1		1	1	1
A'B (2,3)		1	1		

Quine McCluskey Method

Group	Minterms	A	B	C	D	E	F	G
2	20	0	0	1	0	1	0	0
3	28	0	0	1	1	1	0	0
	52	0	1	1	0	1	0	0
4	60	0	1	1	1	1	0	0

• $F(A, B, C, D, E, F, G) = \Sigma(20, 28, 52, 60)$

• $A'CEF'G'$

Group	Pair	A	B	C	D	E	F	G
2	(20,28)	0	0	1	-	1	0	0
	(20,52)	0	-	1	0	1	0	0
3	(28,60)	0	-	1	1	1	0	0
	(52,60)	0	1	1	-	1	0	0

Group	Quad	A	B	C	D	E	F	G
2	(20,28,52,60)	0	-	1	-	1	0	0

Other gate minimisation methods

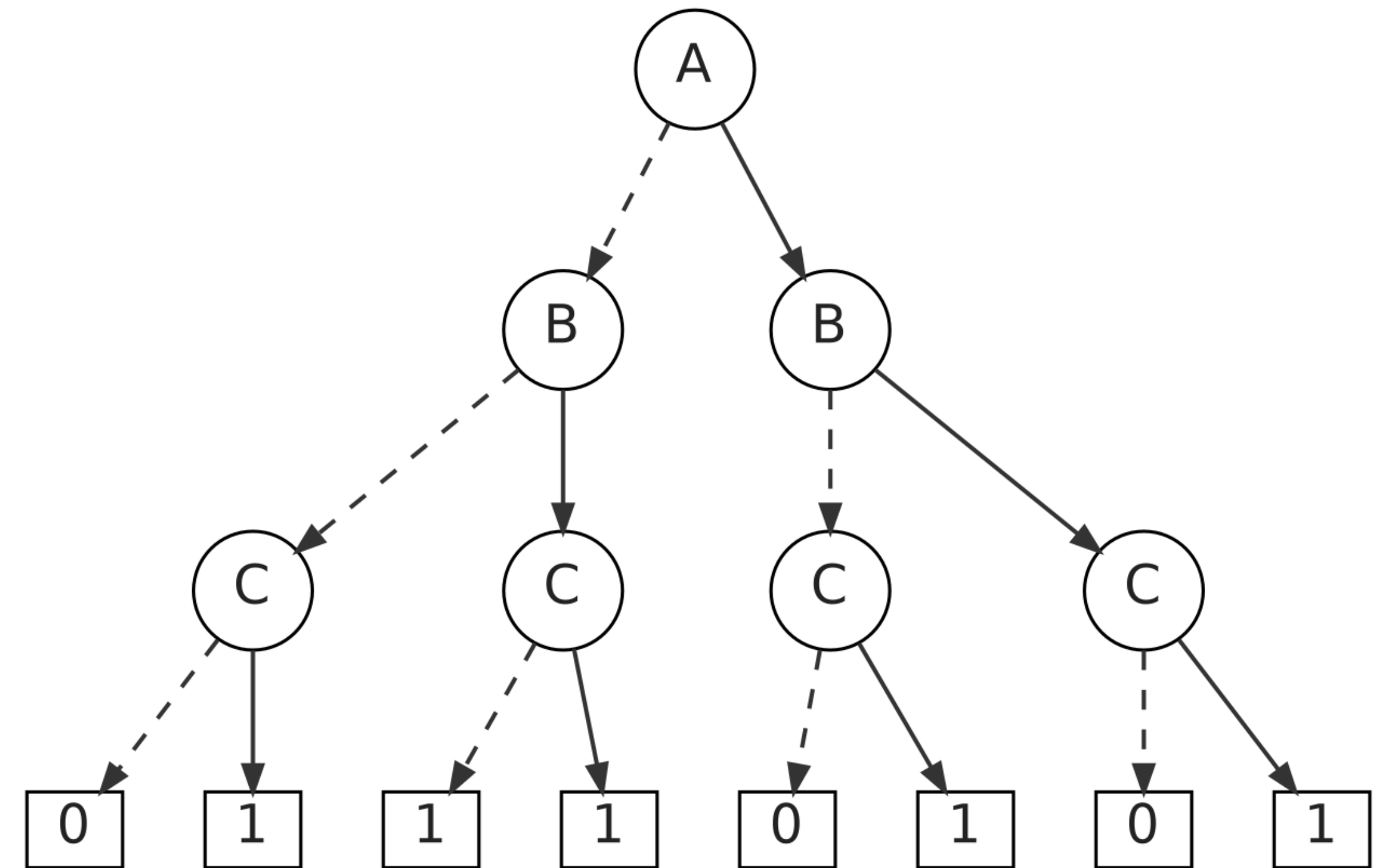
- Both K-map and Quine-McCluskey guarantee minimal function
- Slow because they work at the level of minterms (2^n for n variables)
- More scalable methods that do not guarantee minimal function:
 - Espresso heuristic optimiser
 - Binary-decision diagram

Shannon expansion

- Any boolean function $f(x, y, z)$ can be written as $x \cdot f(1, y, z) + x' \cdot f(0, y, z)$
- $F = A'B'C' + B'CD' + A'BCD' + AB'C'$
- $F = A'B'C' + (A' + A)B'CD' + A'BCD' + AB'C'$
- $F = A'B'C' + A'B'CD' + AB'CD' + A'BCD' + AB'C'$
- $F = A'(B'C' + B'CD' + BCD') + A(B'CD' + B'C')$
- $F = A' \cdot F(0, B, C, D) + A \cdot F(1, B, C, D)$

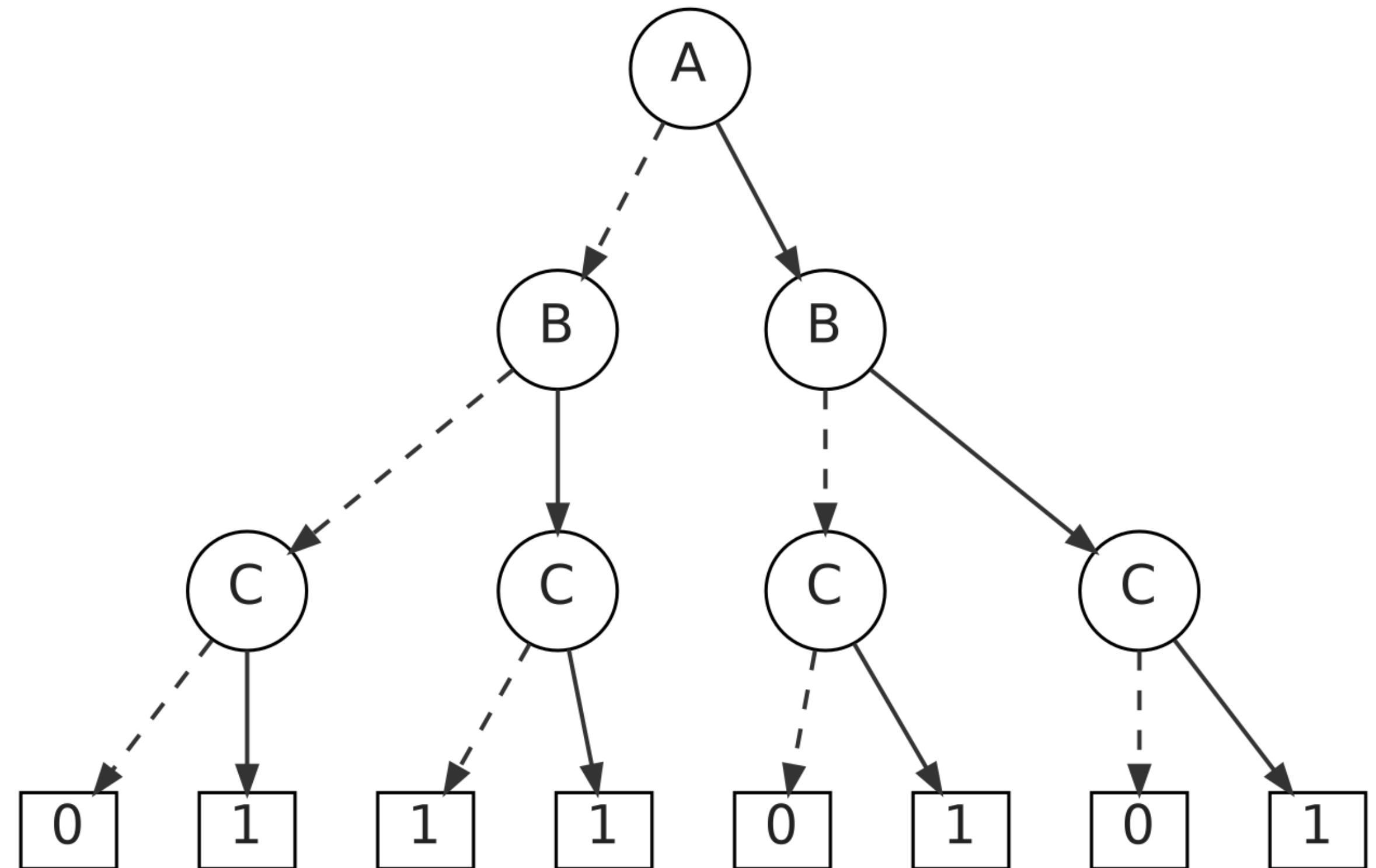
Binary decision diagrams (BDDs)

- $F(A, B, C) = \Sigma(1,2,3,5,7)$
- Binary tree. Each node is a binary variable. If the binary variable is true, follow the solid edge, otherwise follow the dotted edge
- Each leaf is one minterm. For example, the first minterm m_0 follows the path $A'B'C'$



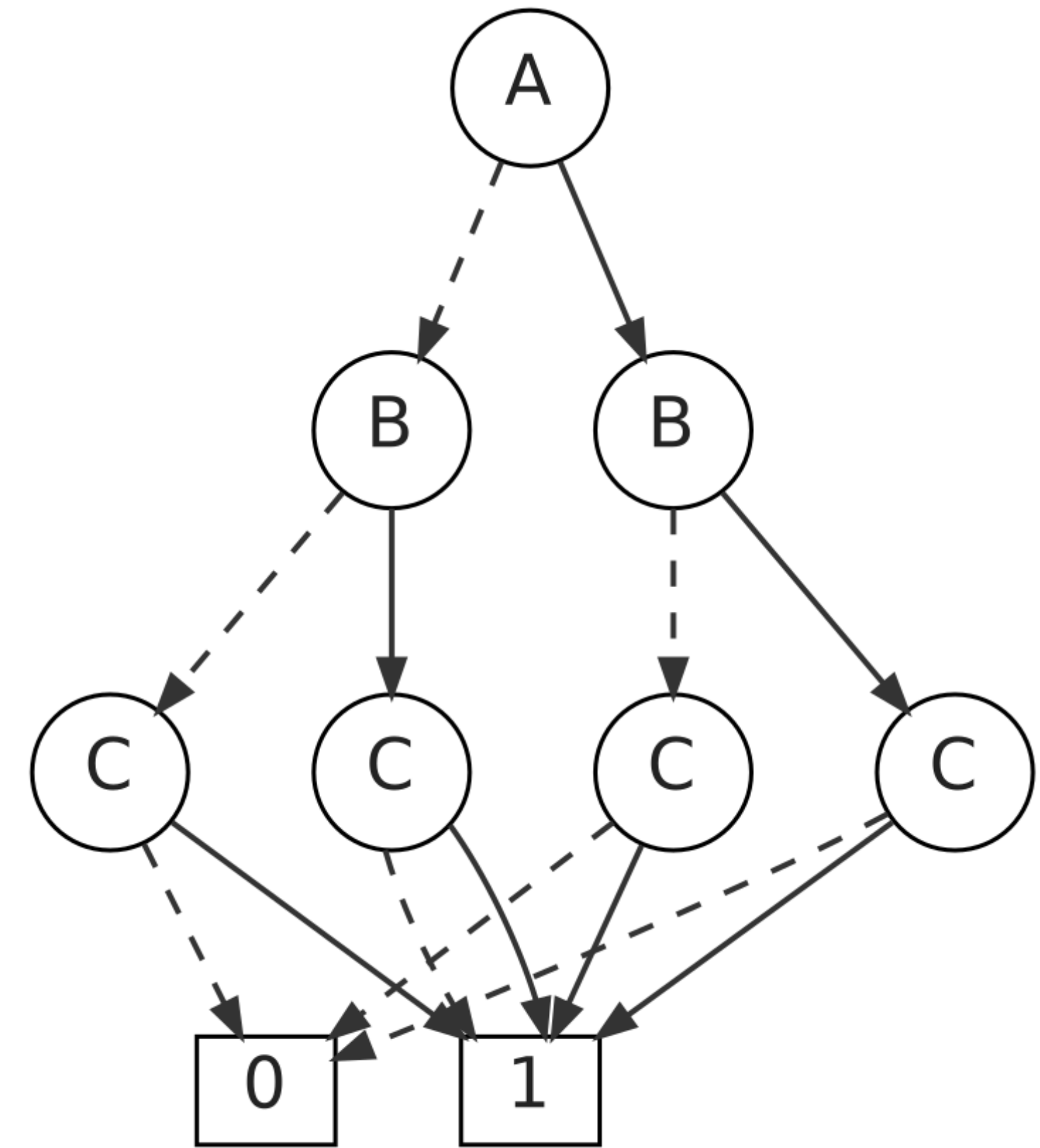
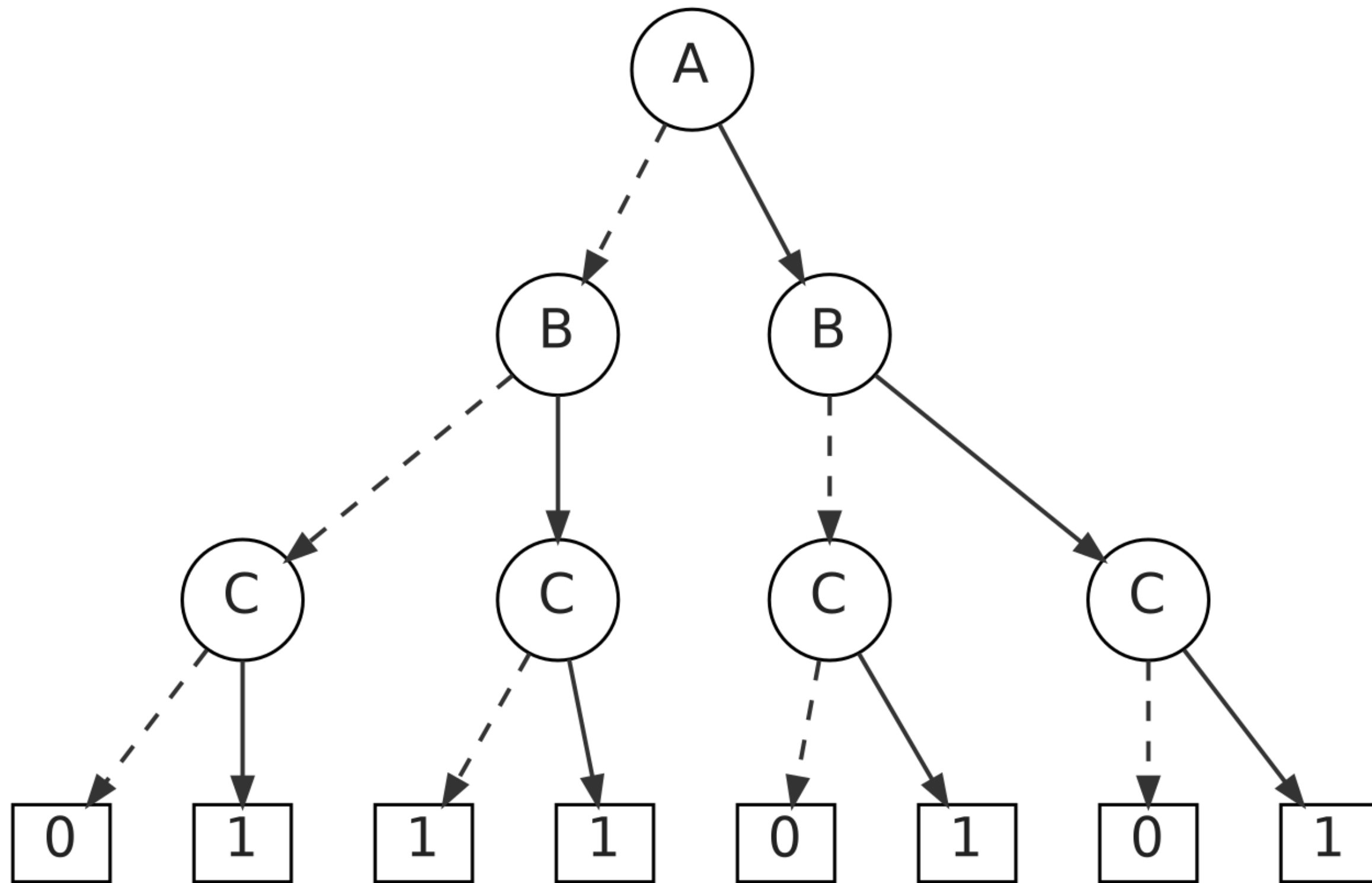
Reducing BDD

- Step 1: Merge all 0s. Merge all 1s
- Step 2: Eliminate nodes where both children are identical
- Step 3: Merge redundant nodes
- Keep applying Steps 2 and 3 until convergence



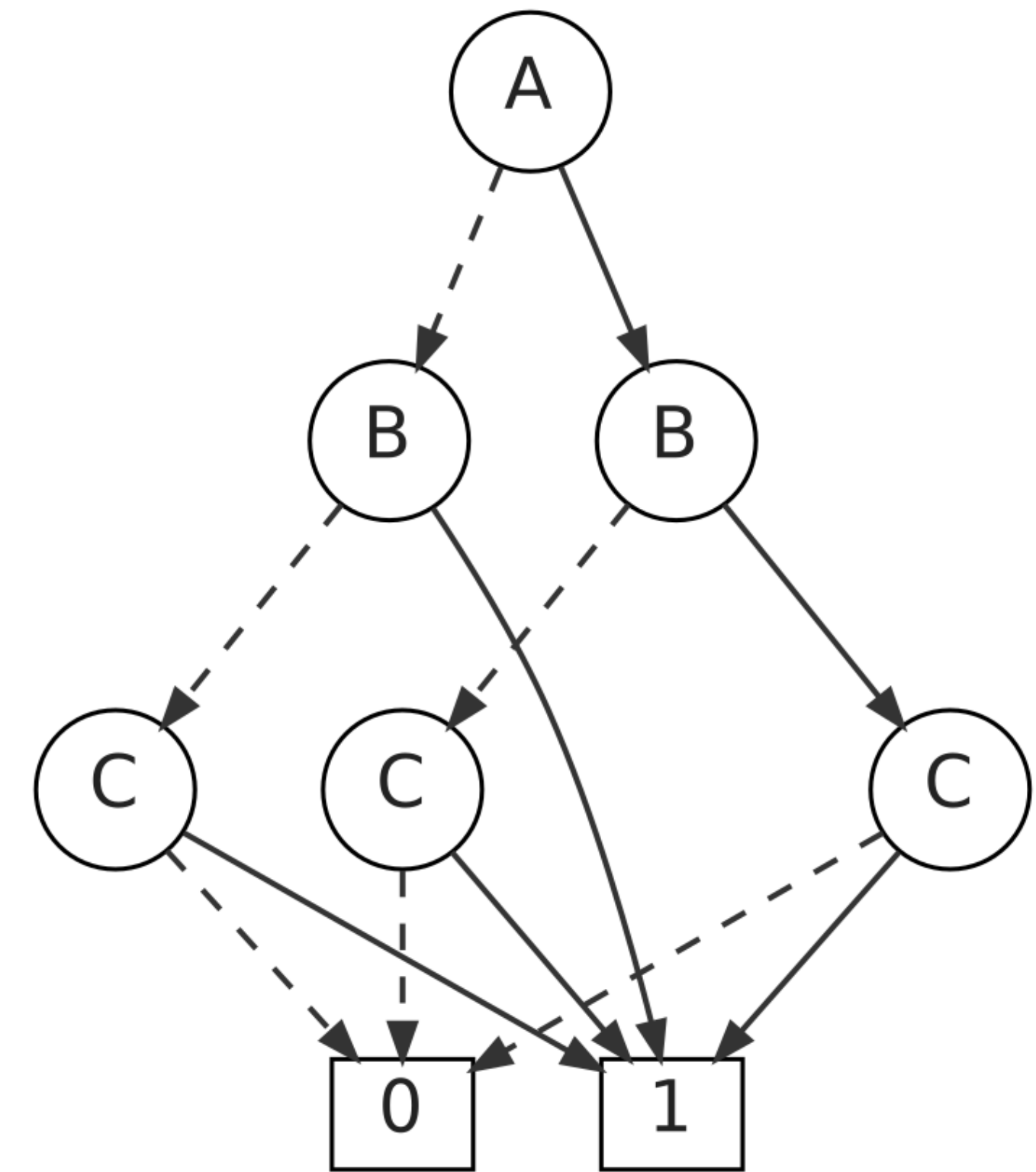
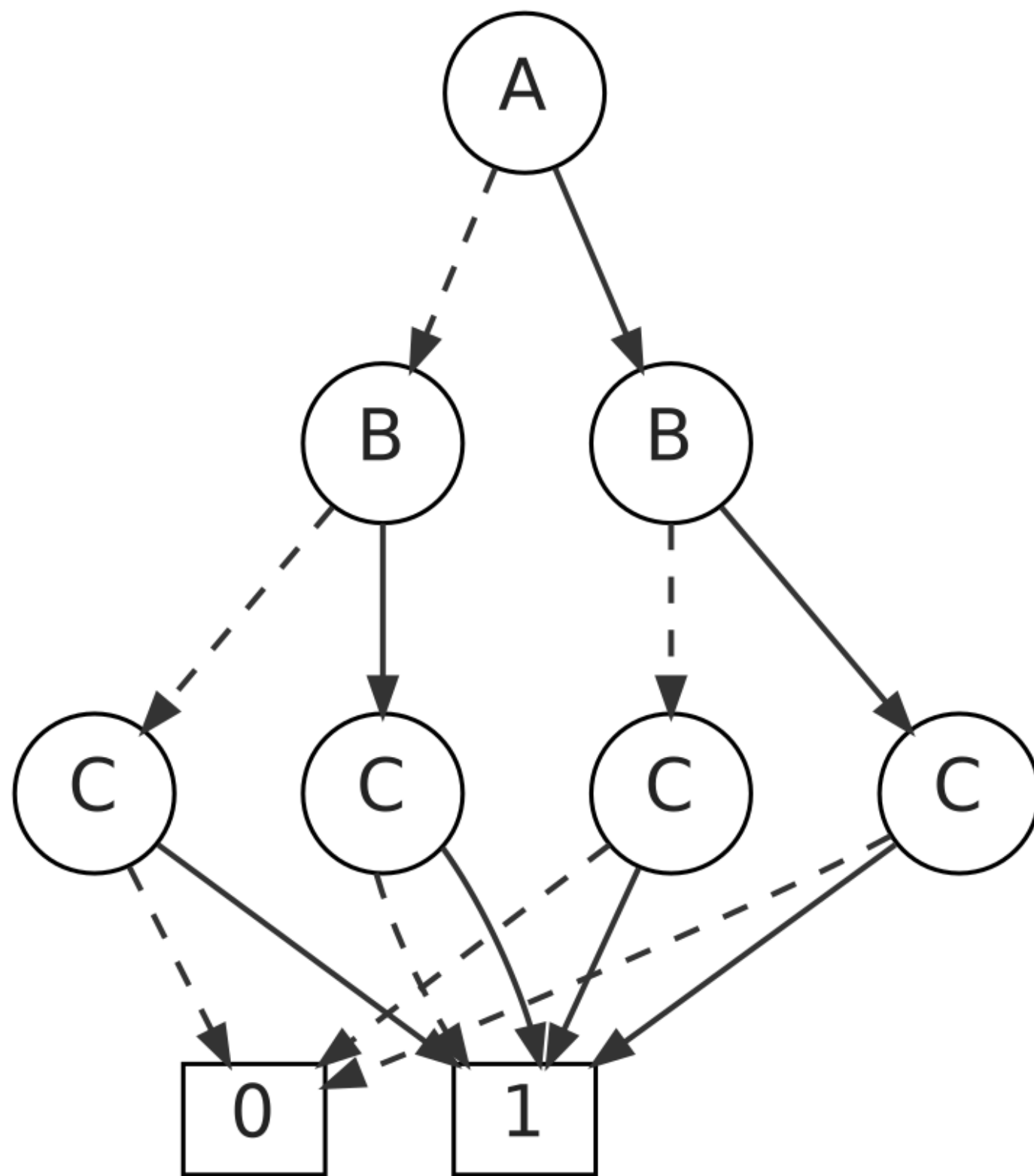
Reducing BDD

Step 1: Merge all 0s. Merge all 1s



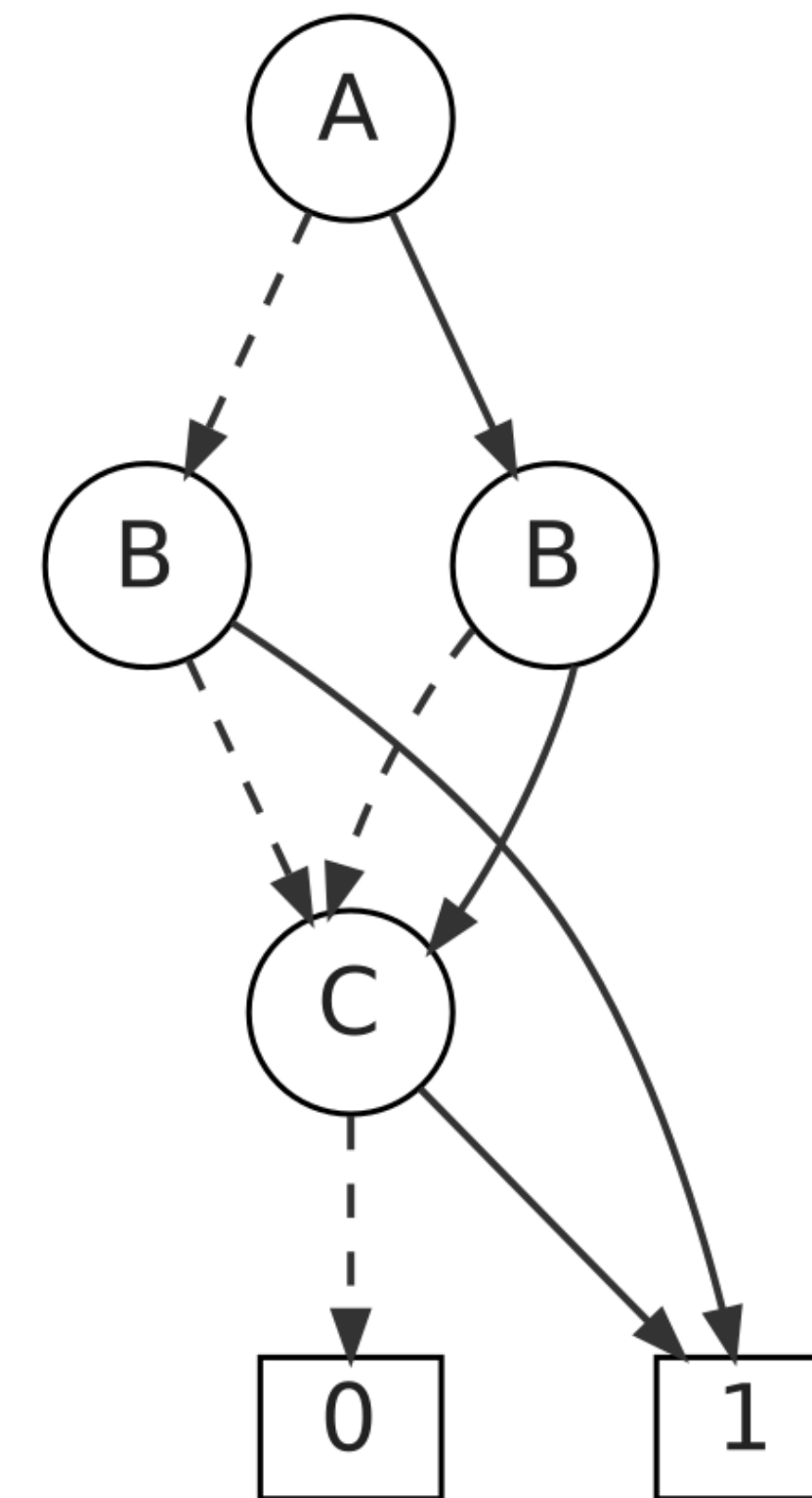
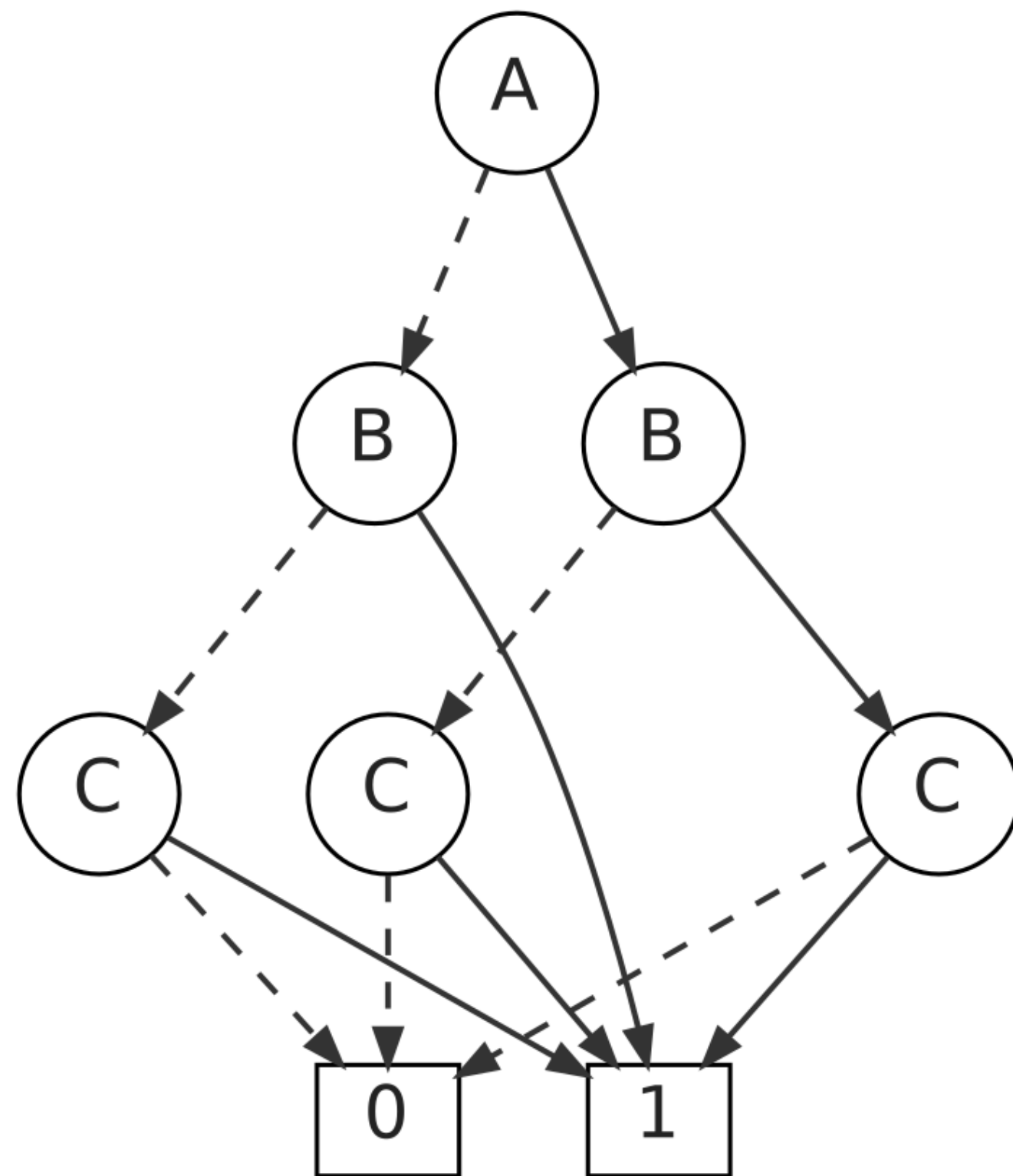
Reducing BDD

Step 2: Eliminate nodes where both children are identical



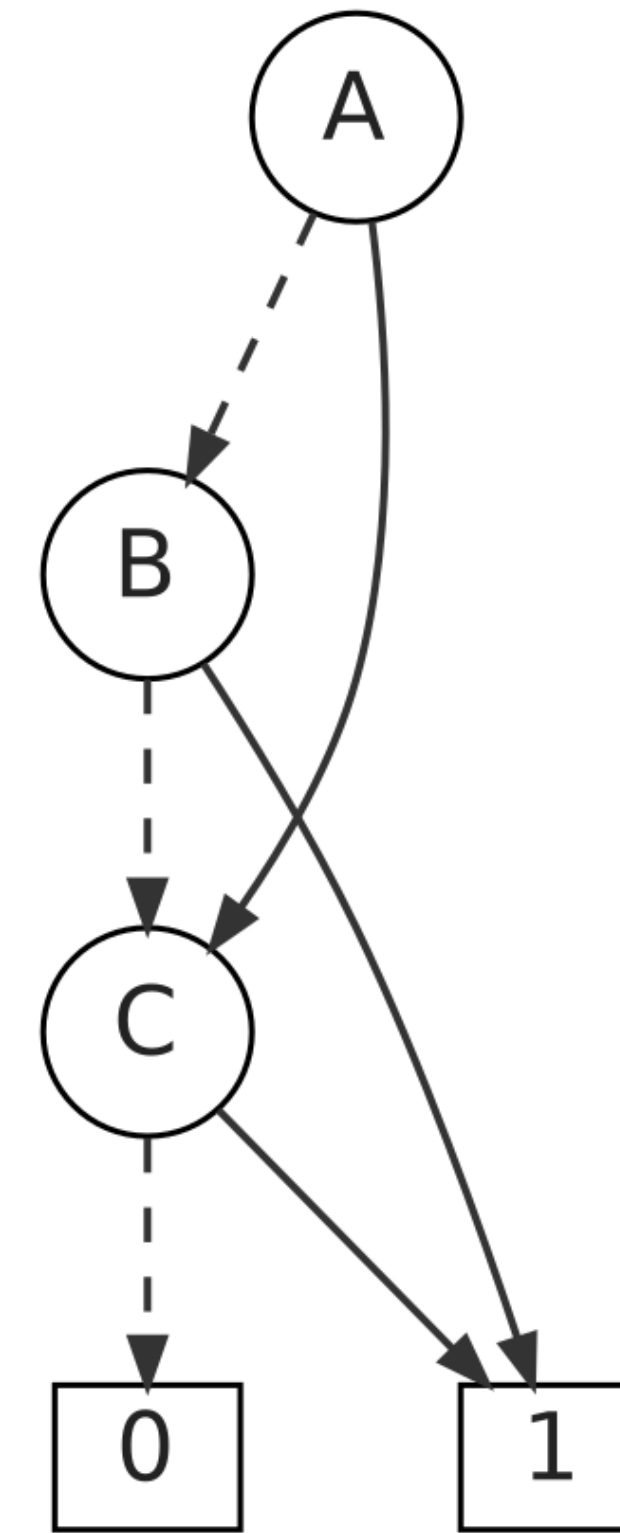
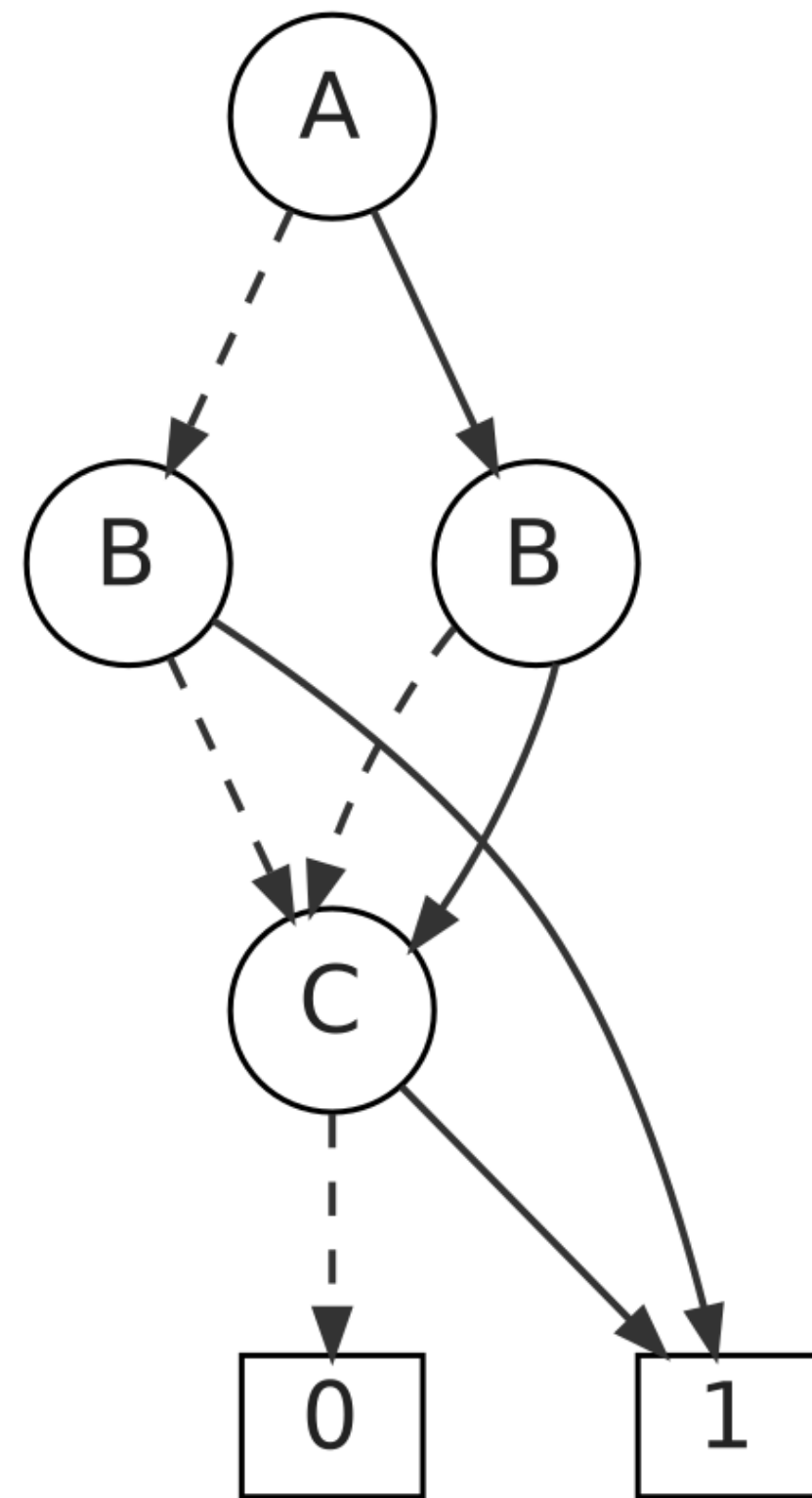
Reducing BDD

Step 3: Merge redundant nodes



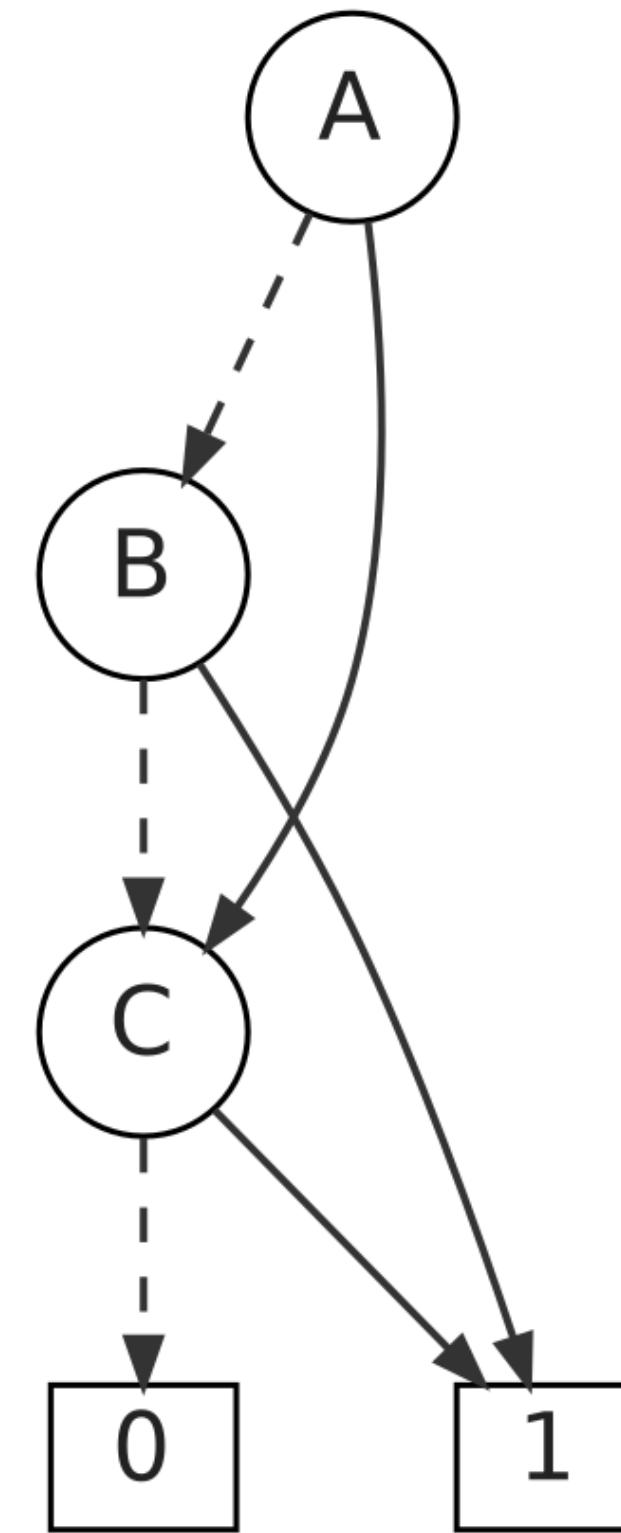
Reducing BDD

Keep applying Steps 2 and 3 until convergence



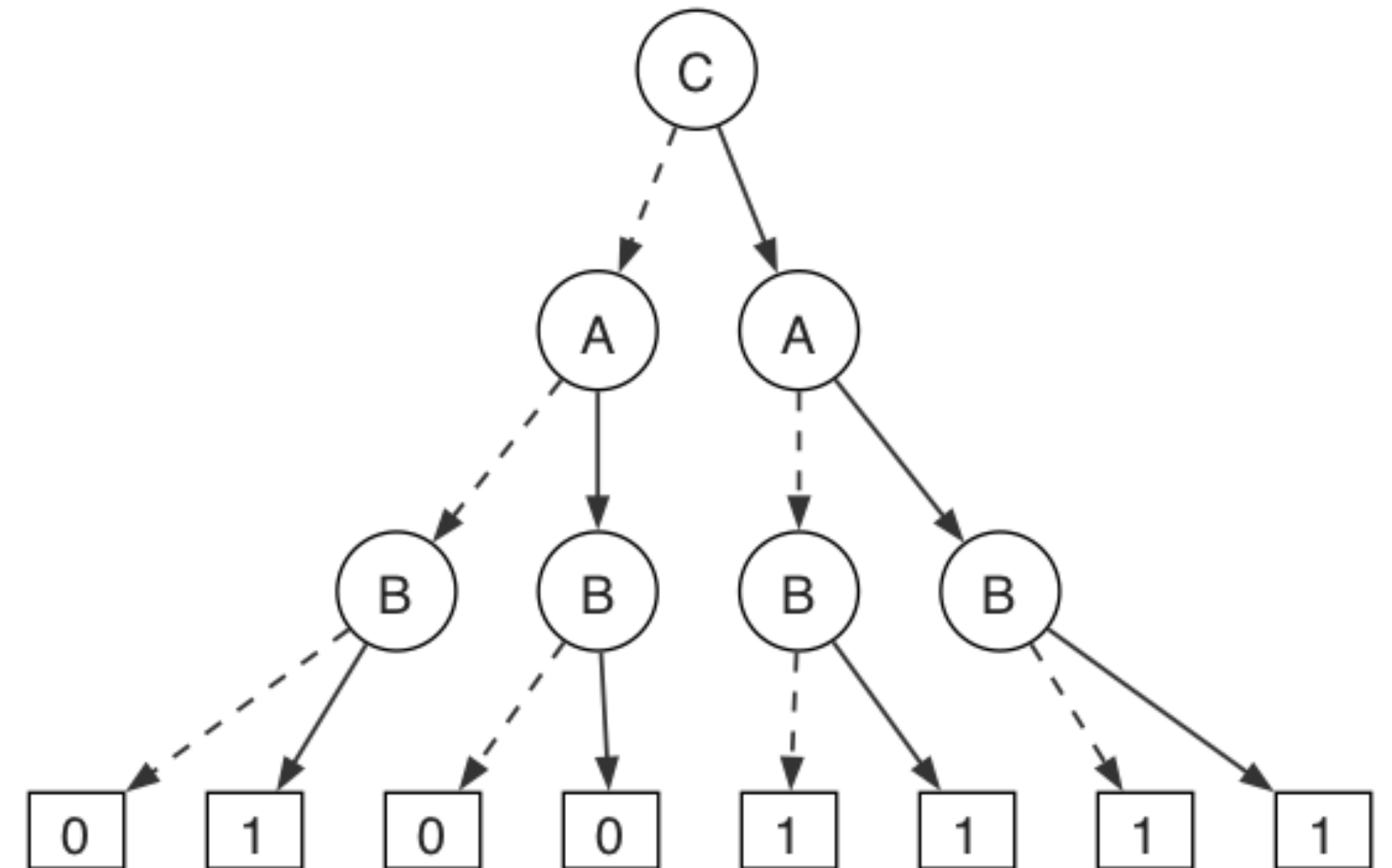
Simplifying functions with BDD

- Write the paths to 1 in an OR
- $AC + A'B + A'B'C$
- $AC + A'(B + B'C)$
- $AC + A'(B + C)$
- $AC + A'B + A'C$
- $A'B + C$



Different orders give different simplifications!

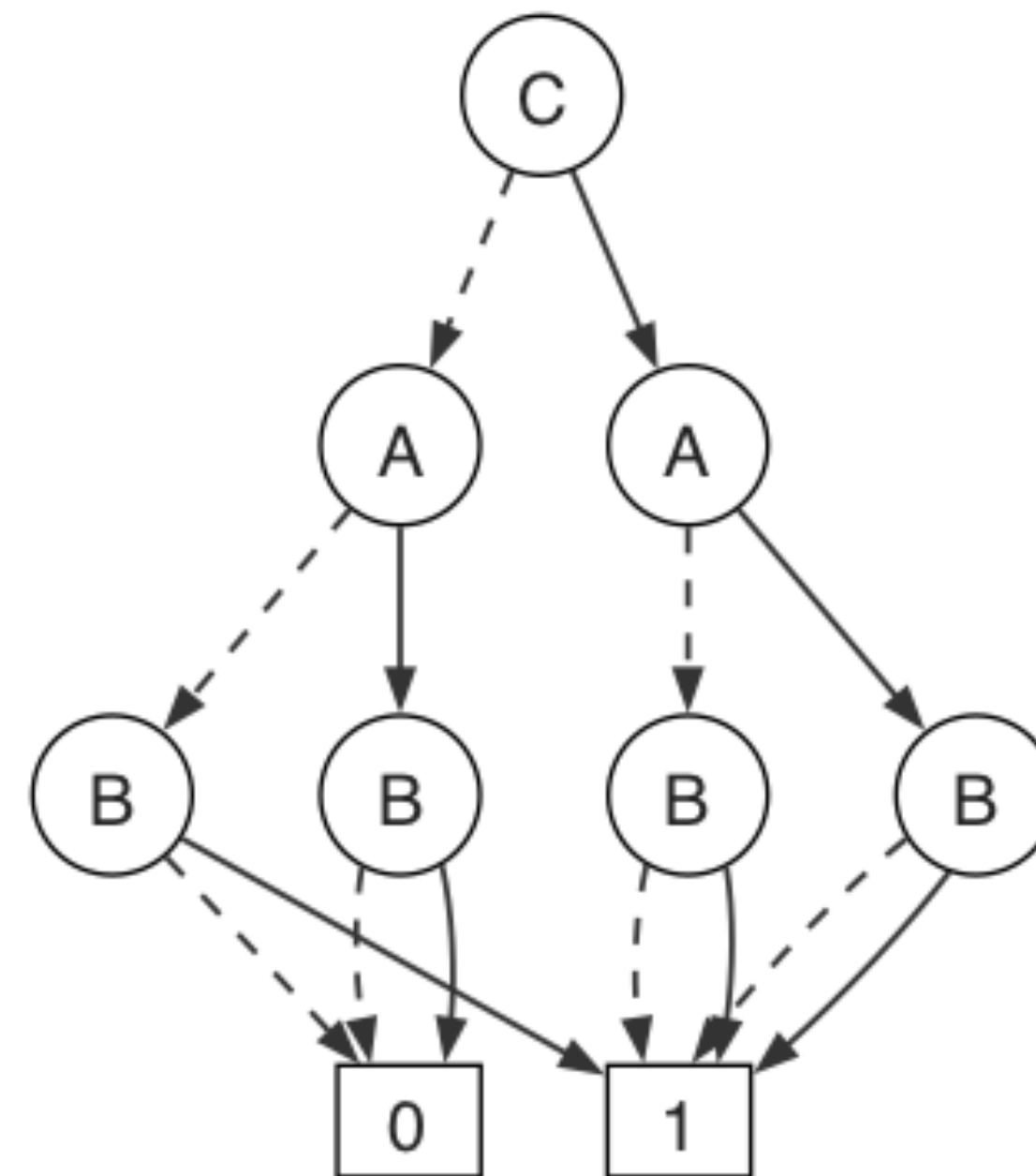
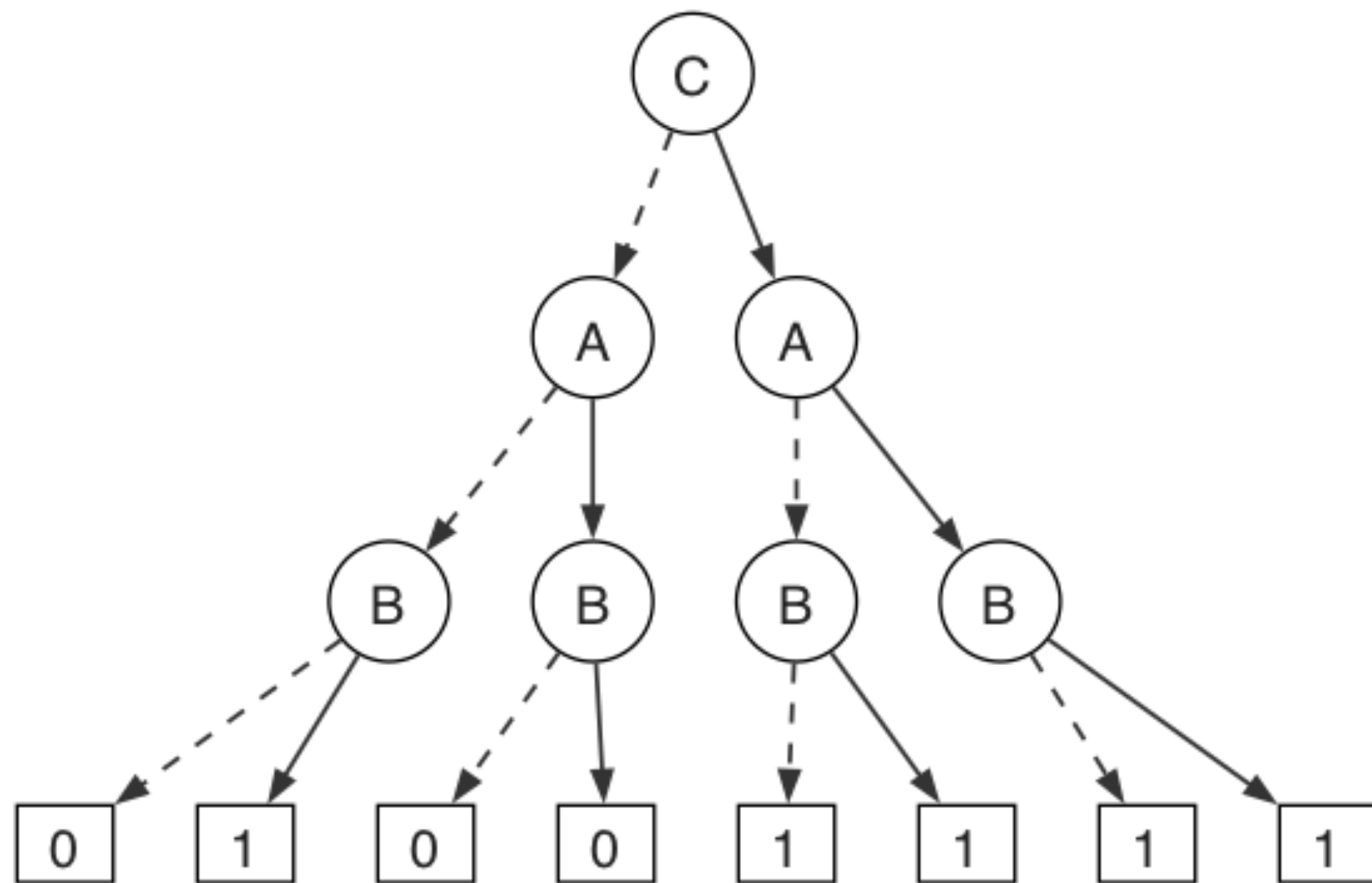
- $F(A, B, C) = \Sigma(1,2,3,5,7)$
- Step 1: Merge all 0s. Merge all 1s
- Step 2: Eliminate nodes where both children are identical
- Step 3: Merge redundant nodes
- Keep applying Steps 2 and 3 until convergence



Different orders give different simplifications!

Step 1: Merge all 0s. Merge all 1s

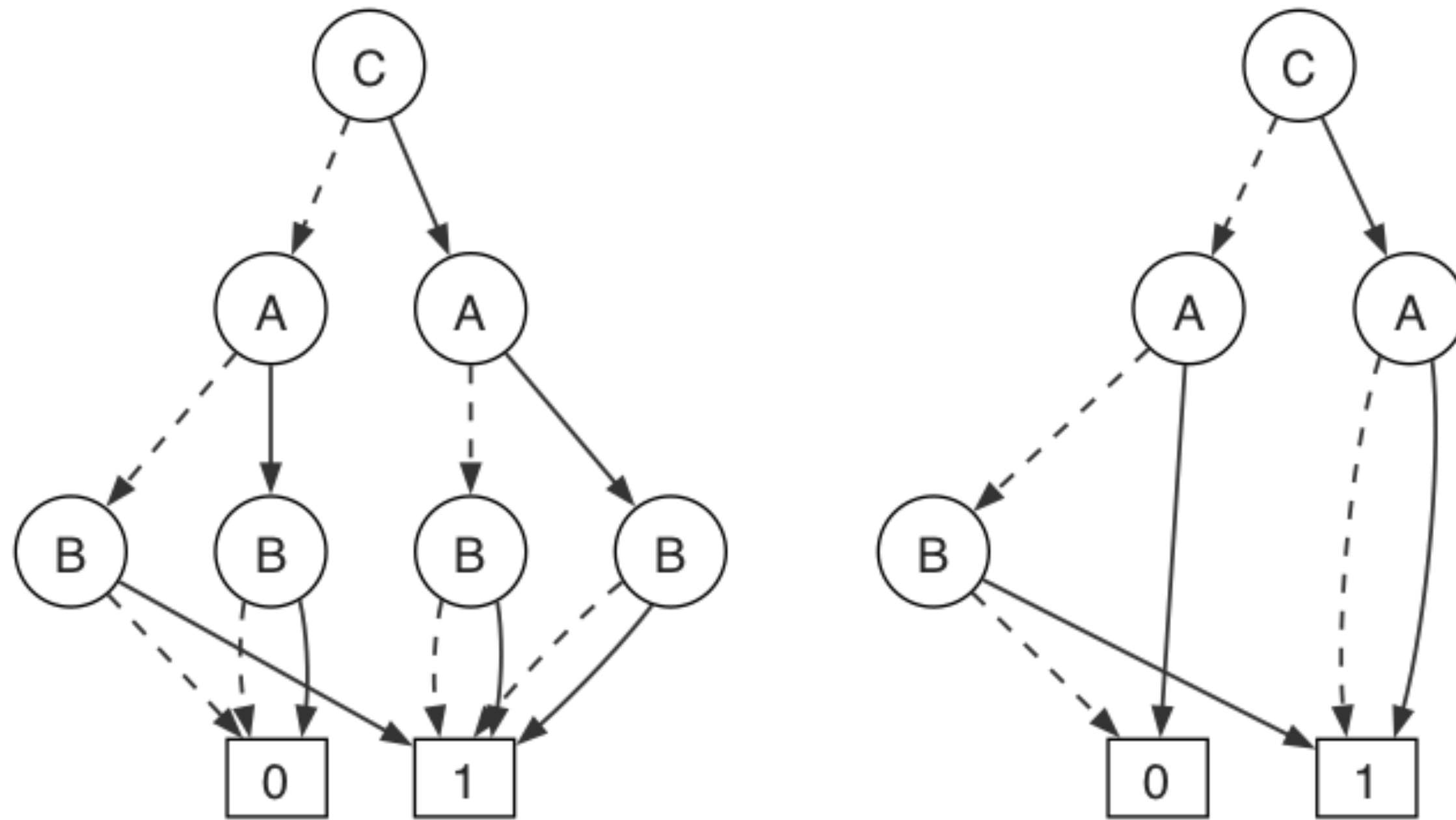
- $F(A, B, C) = \Sigma(1, 2, 3, 5, 7)$



Different orders give different simplifications!

Step 2: Eliminate nodes where both children are identical

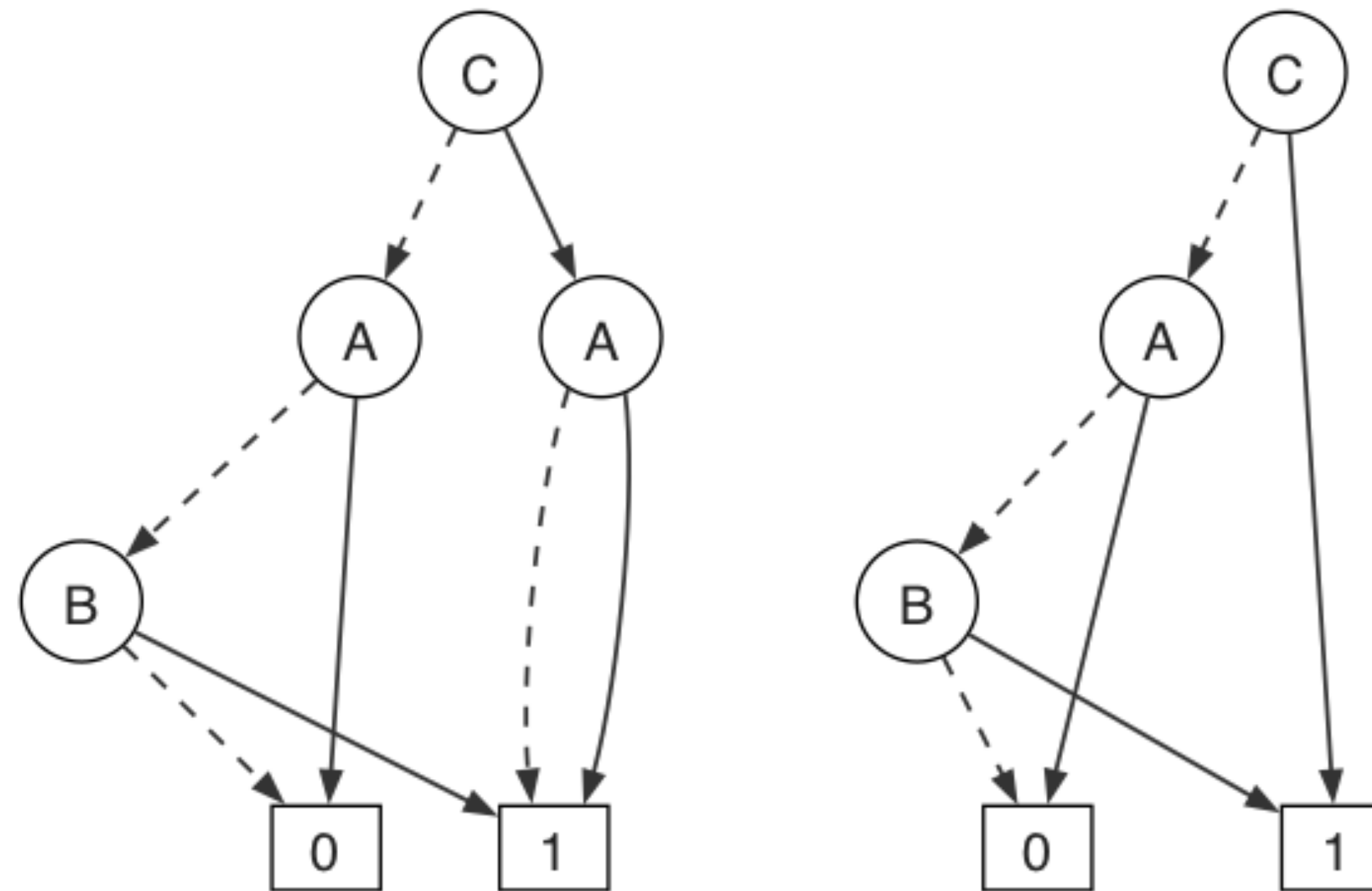
- $F(A, B, C) = \Sigma(1, 2, 3, 5, 7)$



Different orders give different simplifications!

Keep applying Steps 2 and 3 until convergence

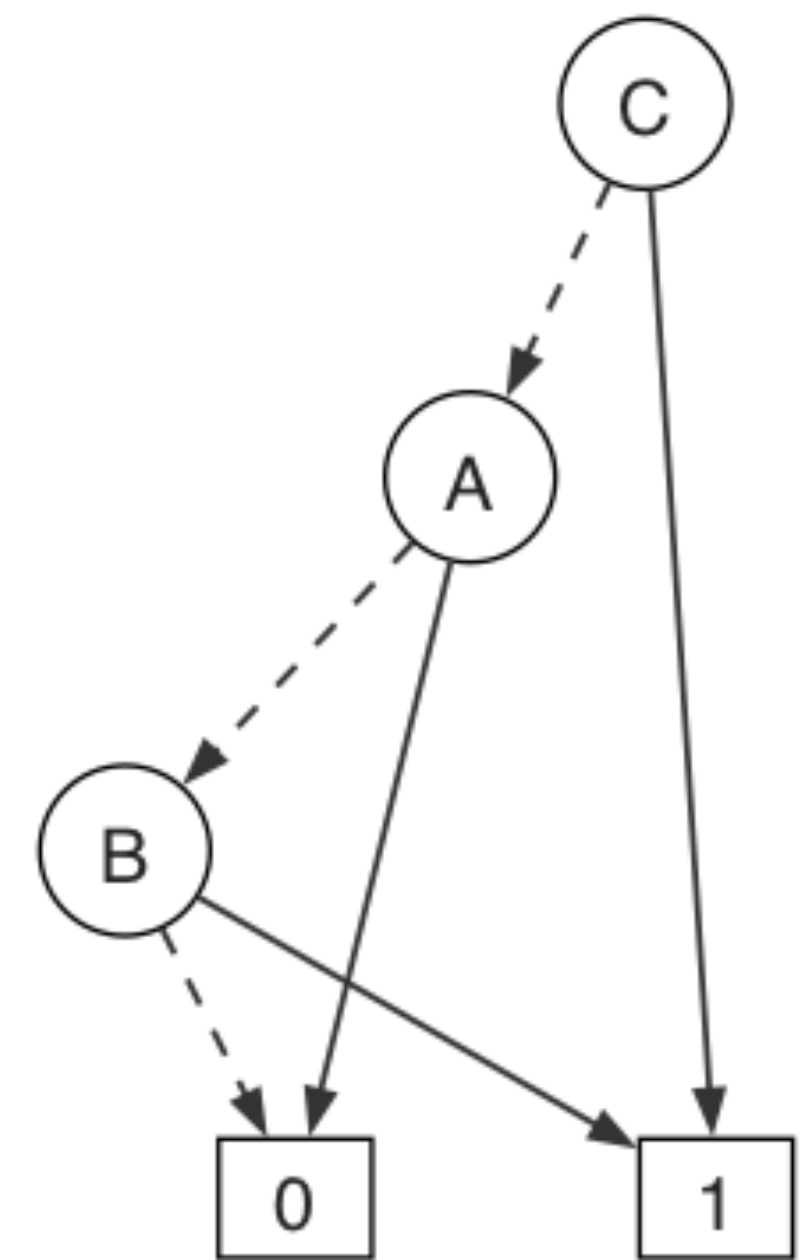
- $F(A, B, C) = \Sigma(1, 2, 3, 5, 7)$



Different orders give different simplifications!

Keep applying Steps 2 and 3 until convergence

- $F(A, B, C) = \Sigma(1, 2, 3, 5, 7)$
- Write the paths to 1 in an OR
- $C + C'A'B$
- $C + A'B$



Reducing BDD

- $F(w, x, y, z) = \Sigma(0,1,3,8,9,10,11,12,13,14,15)$
- $w + w'x'y' + w'x'yz$
- $w + x'y' + x'yz$
- $w + x'y' + x'z$